

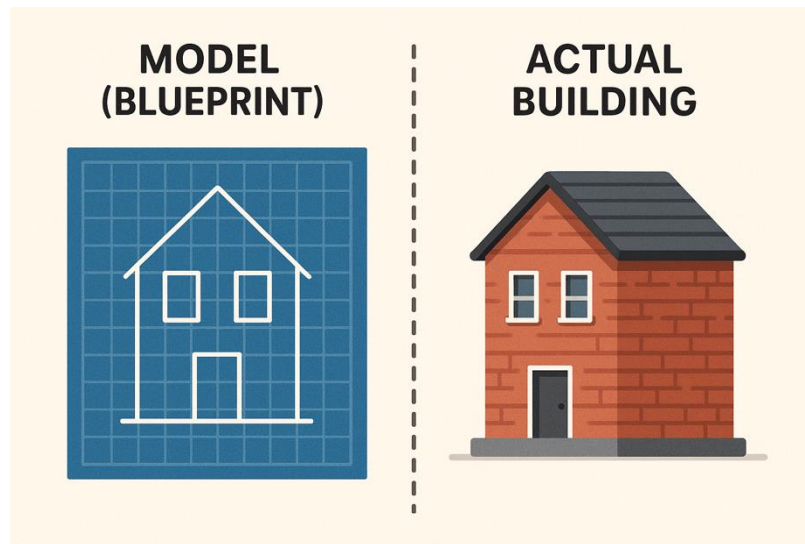
Week 11

Behavioral Modeling



Models

- A *model* is a description of something
 - “a pattern for something to be made” (Merriam-Webster)



- model $\neq$ thing that is modeled
 - The Map is Not The Territory

Models

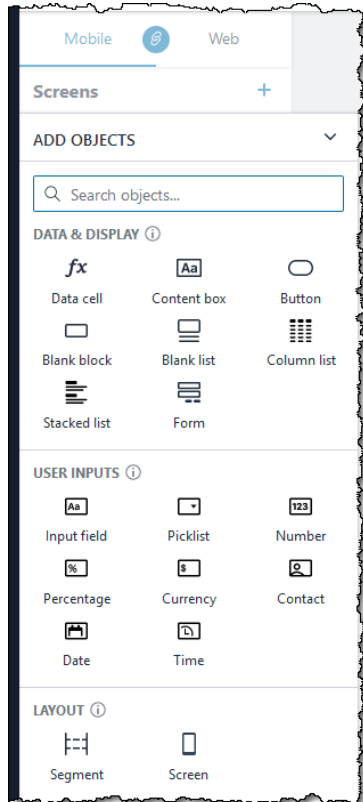
- How do we model?
- Modeling Maturity Level
 - Level 0: No specification
 - Level 1: Textual
 - Level 2: Text with Diagrams
 - Level 3: Models with Text
 - Level 4: Precise Models

Modeling Languages

Largely classified into two types

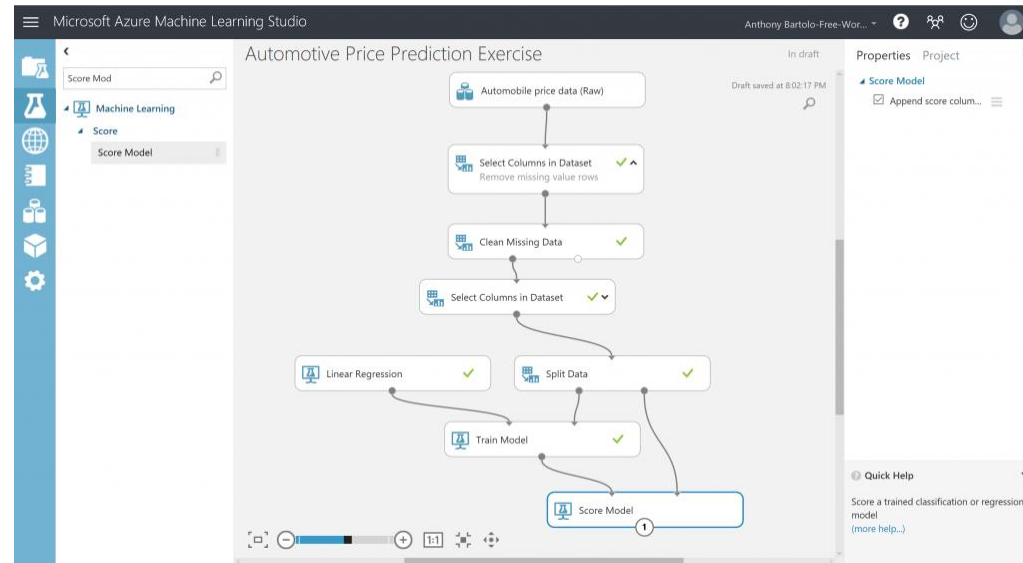
- Domain-Specific Languages (DSLs)
 - Languages designed to model a certain domain
 - Examples: HTML, SQL, ThingML, etc.
- General Purpose Modeling Languages (GPLs)
 - Languages can be applied to any domain for modeling
 - Examples: **UML**, SysML, XML, etc.

Some Examples



AWS Honeycode

(Moving to AWS Amplify)

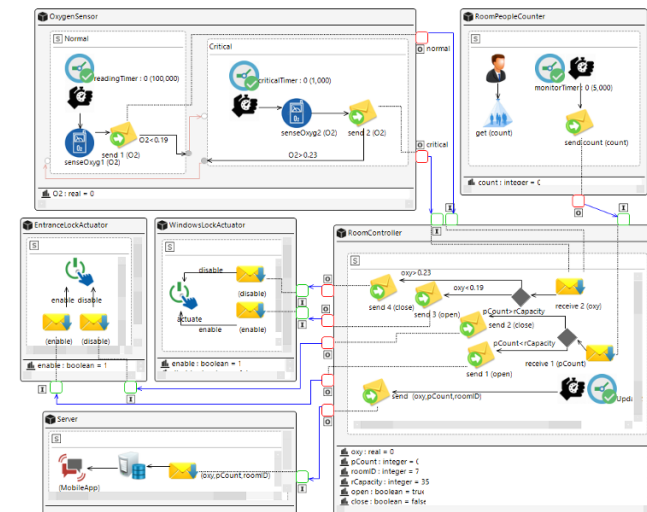


Azure Machine Learning Studio

```
configuration Robot
{
    instance robot : RobotControl
    instance sdist : DistanceSensor
    instance scoll : CollisionSensor
    instance motion : MotionControl
    instance left_wheel : WheelControl
    instance right_wheel : WheelControl

    connector robot.rangefinder => sdist.data
    connector robot.bumper => scoll.data
    connector robot.platform => motion.ctrl
    connector motion.left => left_wheel.ctrl
    connector motion.right => right_wheel.ctrl
}
```

ThingML modeling



CAPS modeling for IoT

In the Context of Our Course
(GPL – UML)

Unified Modeling Language (UML): Brief History



- No common language to model until 1996
- UML developed by industry consortium in 1997
 - Introduction of OOP in IT dates back to 1960's
 - Required a standard representation: **OMG**
 - Three Amigos: Grady Booch, Ivar Jacobson and James Rumbaugh
- Based on multiple prior visual modeling languages
- Goal was to have a single language that could cover large number of SE tasks
- Current version of UML: 2.5.1 (as of Dec 2017)

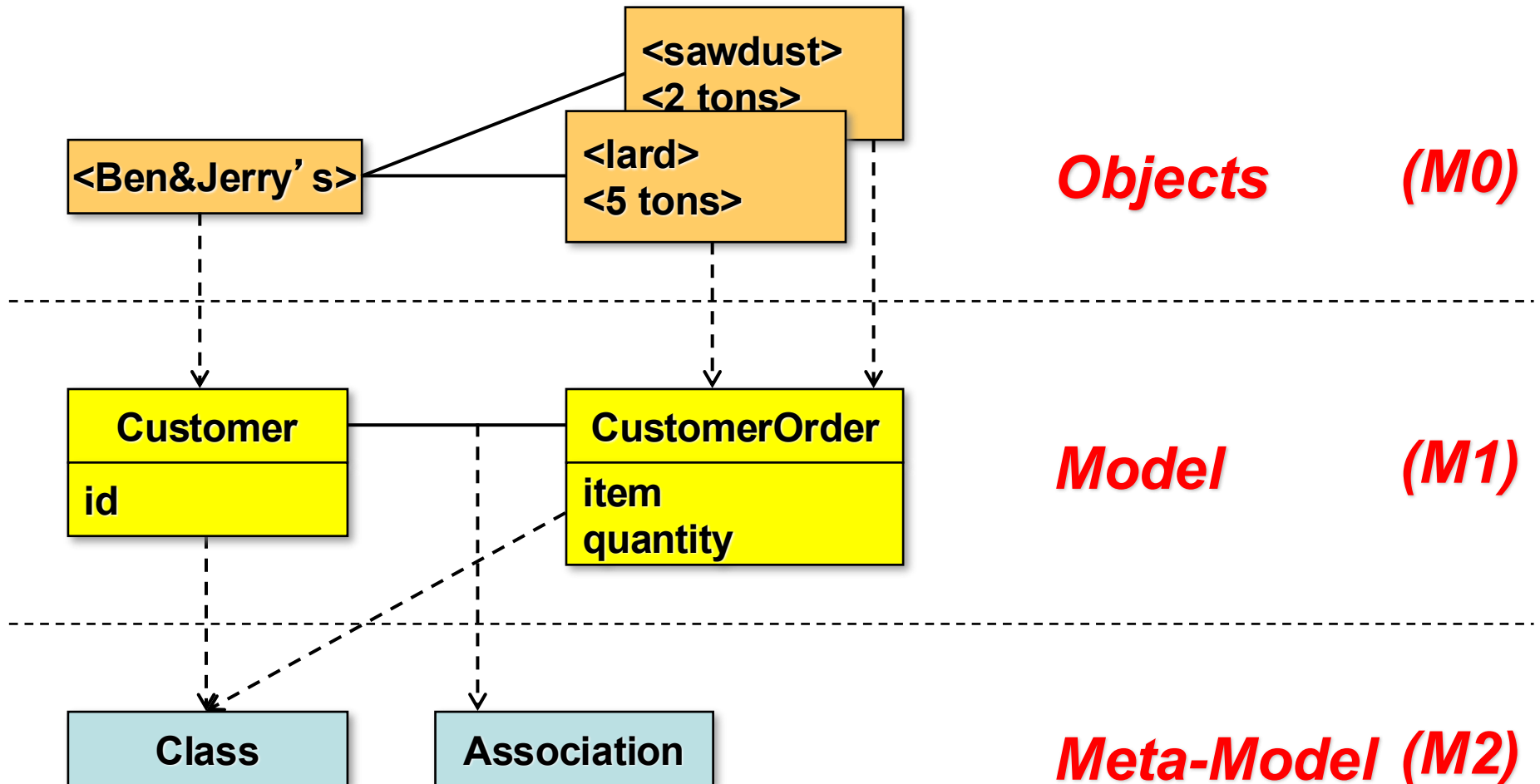
What UML is not?

- Not an OO Method or Process
- Not a visual programming language
- Not a tool specification

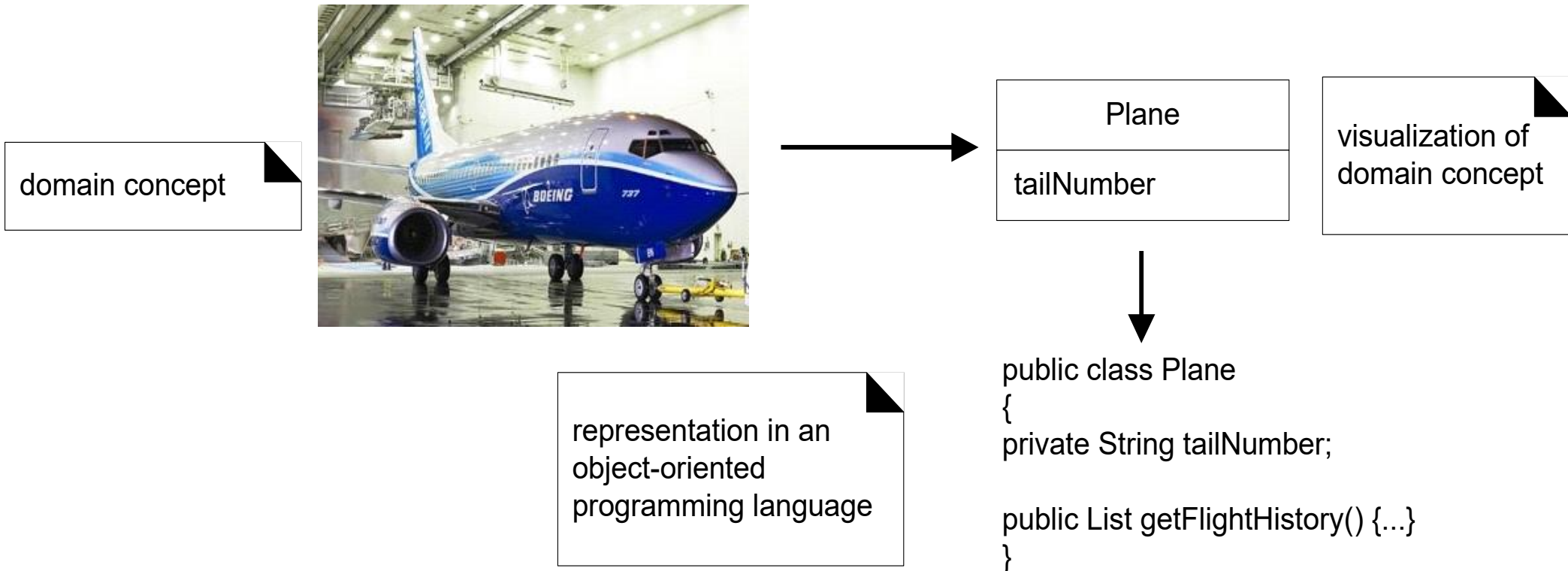


Models and Meta-Models

Meta-models are simply Models of Models

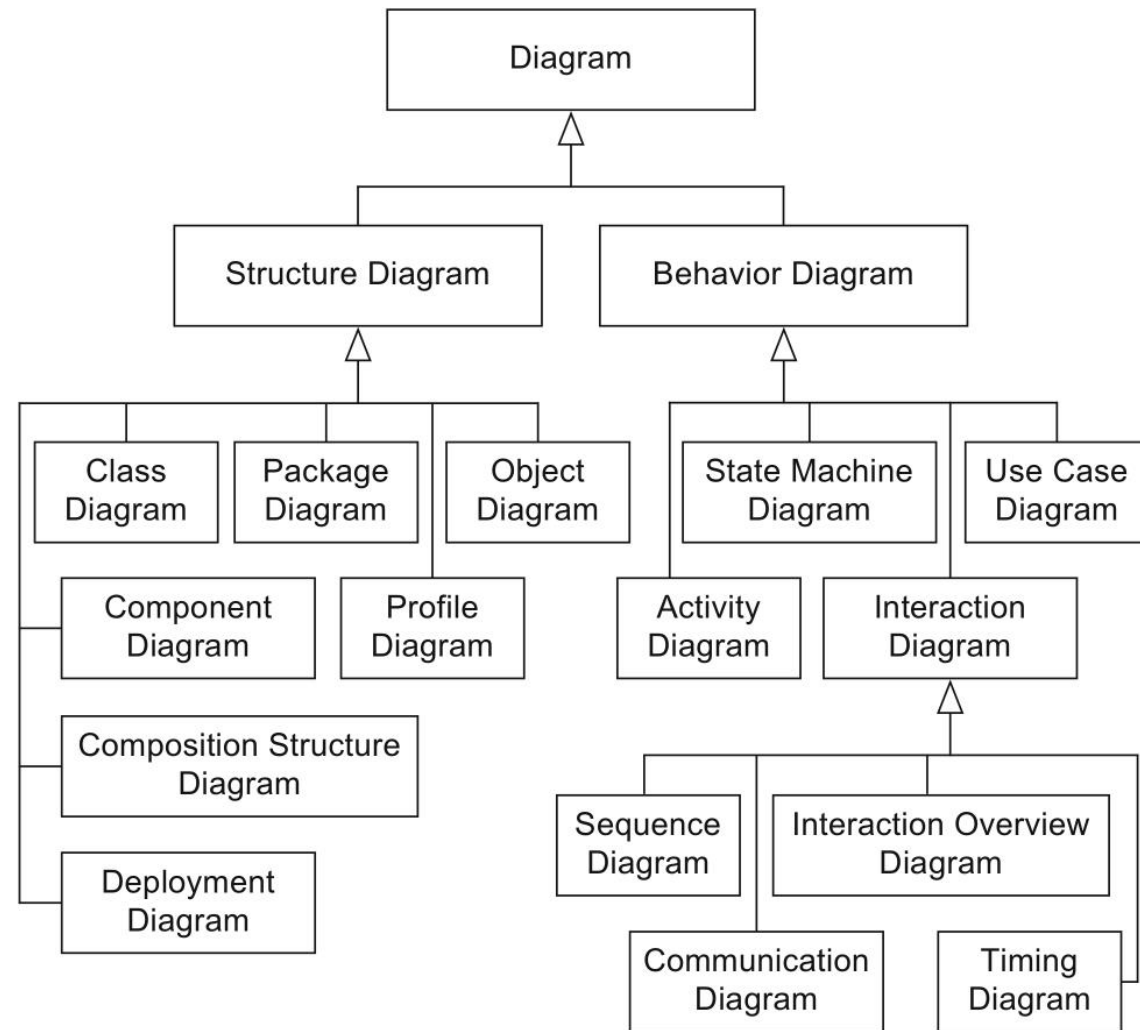


Domain concept
vs.
design representation of domain concept
vs.
code representation of domain concept



UML Diagrams

- 14 different diagrams
- Structure diagrams for capturing static aspects of system
- Behavior diagrams for capturing dynamic aspect of system



Static Vs Dynamic Models

Static Model

- Describes the static structure of a system
- One of the most common diagrams: *class diagrams*

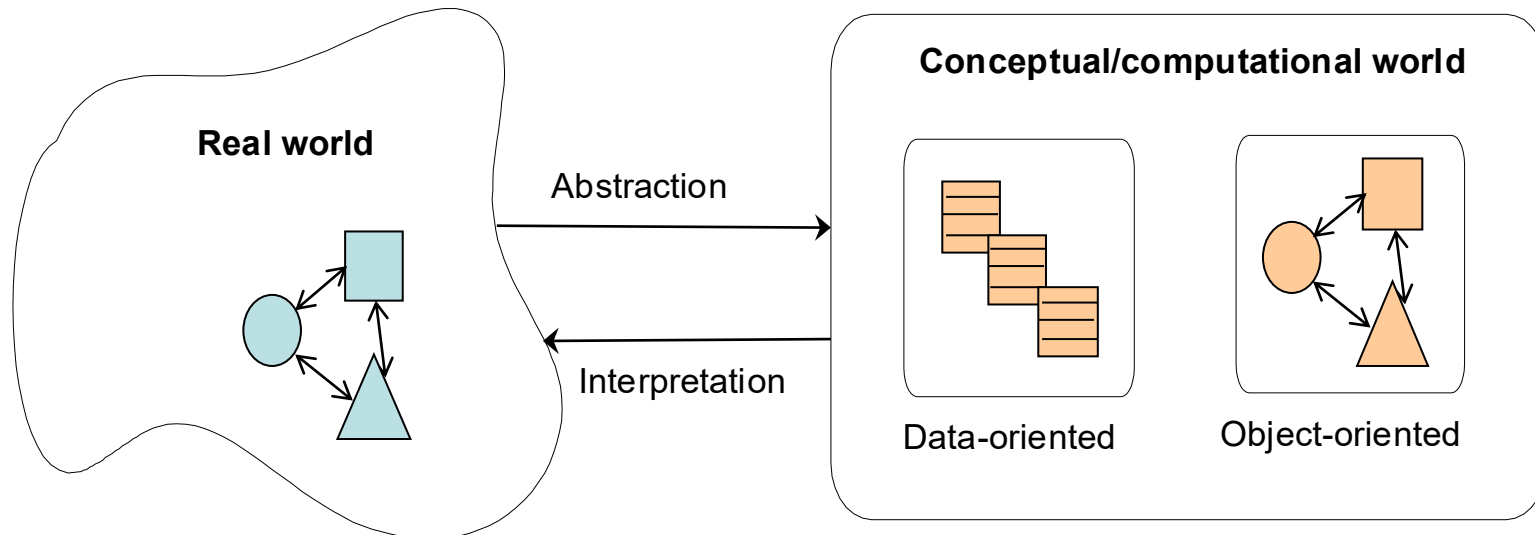
Dynamic Model

- Captures the dynamic behavior of a system
- Developed with help of state chart diagrams, sequence diagrams, etc.



Object-Oriented Modeling

- Uses object-orientation as a basis of modeling
- Models a system as a set of objects that interact with each others
- No semantic gap (or impedance mismatch)
- Seamless development process



Key Ideas of O-O Modeling

- Abstraction
- Encapsulation
- Relationship
 - Association: relationship between objects
 - Inheritance: mechanism to represent similarity among objects
- Object-oriented
 - = object (class) + inheritance + message send

Objects vs. Classes

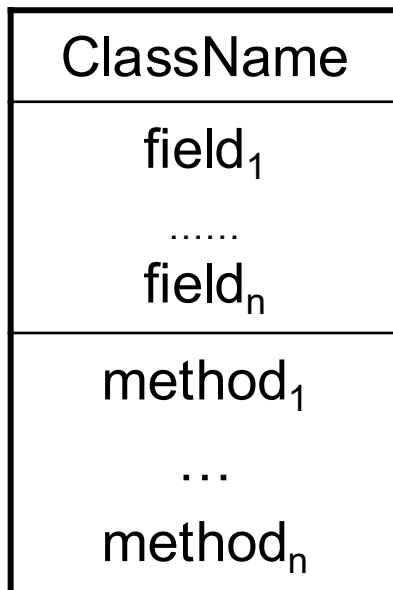
	Interpretation in the Real World	Representation in the Model
Object	An <i>object</i> represents anything in the real world that can be distinctly identified.	An <i>object</i> has an identity, a state, and a behavior.
Class	A <i>class</i> represents a set of objects with similar characteristics and behavior. These objects are called <i>instances</i> of the class.	A <i>class</i> characterizes the structure of states and behaviors that are shared by all of its instances.

UML Class Diagram

- Most common diagram in OO modeling
- Describes the static structure of a system
- Consist of:
 - Nodes representing classes
 - Links representing of relationships among classes
 - Inheritance
 - Association, including aggregation and composition
 - Dependency

Notation for Classes

- The UML notation for classes is a rectangular box with as many as three compartments.



The top compartment show the class name.

The middle compartment contains the declarations of the fields, or *attributes*, of the class.

The bottom compartment contains the declarations of the methods of the class.

Example

Point

Point
x
y
move

Point
- x: int - y: int
+ move(dx: int, dy: int): void

Field and Method Declarations in UML

- Field declarations
 - birthday: Date
 - +duration: int = 100
 - -students[1..MAX_SIZE]: Student
- Method declarations
 - +move(dx: int, dy: int): void
 - +getSize(): int

Visibility	Notation
public	+
protected	#
package	~
private	-

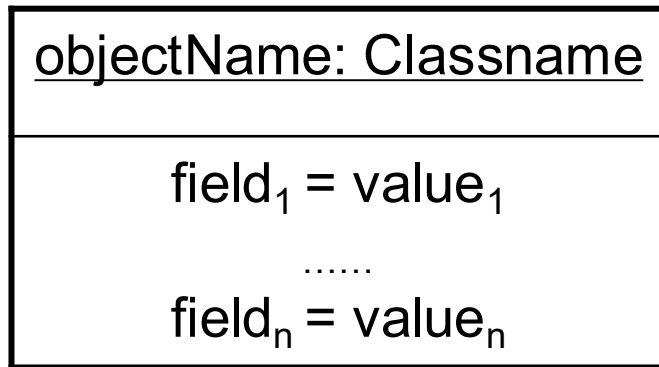
Practice activity

- Model a UML class diagram for the following Java code.

```
class Person {  
    private String name;  
    private Date birthday;  
    public String getName() {  
        // ...  
    }  
    public Date getBirthday() {  
        // ...  
    }  
}
```

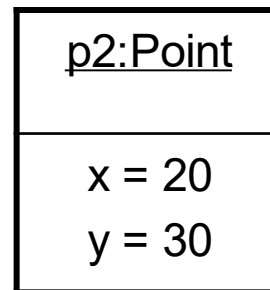
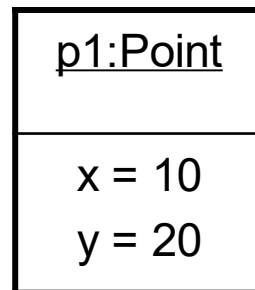
Notation for Objects

Rectangular box with one or two compartments



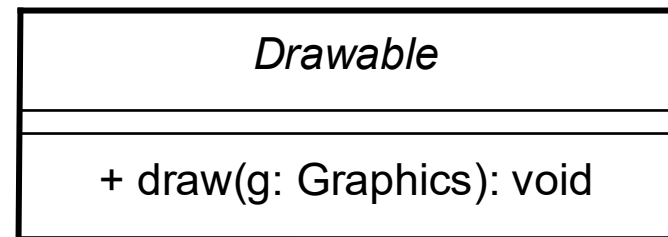
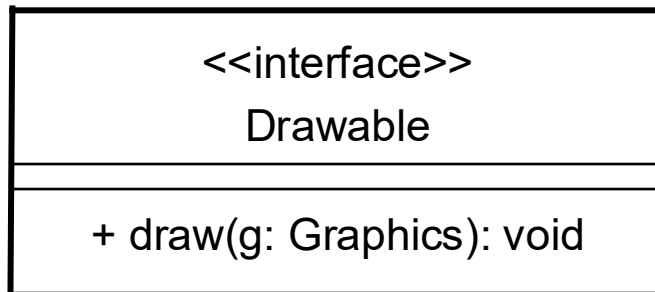
The top compartment shows the name of the object and its class.

The bottom compartment contains a list of the fields and their values.



UML Notation for Interfaces

```
interface Drawable {  
    void draw(Graphics g);  
}
```

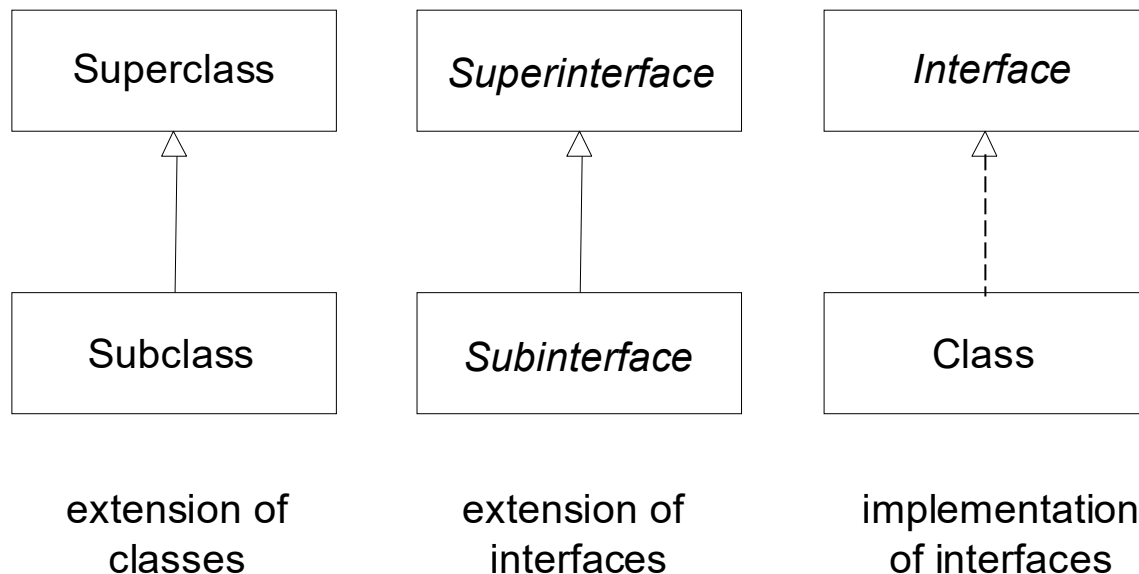


Inheritance in Java

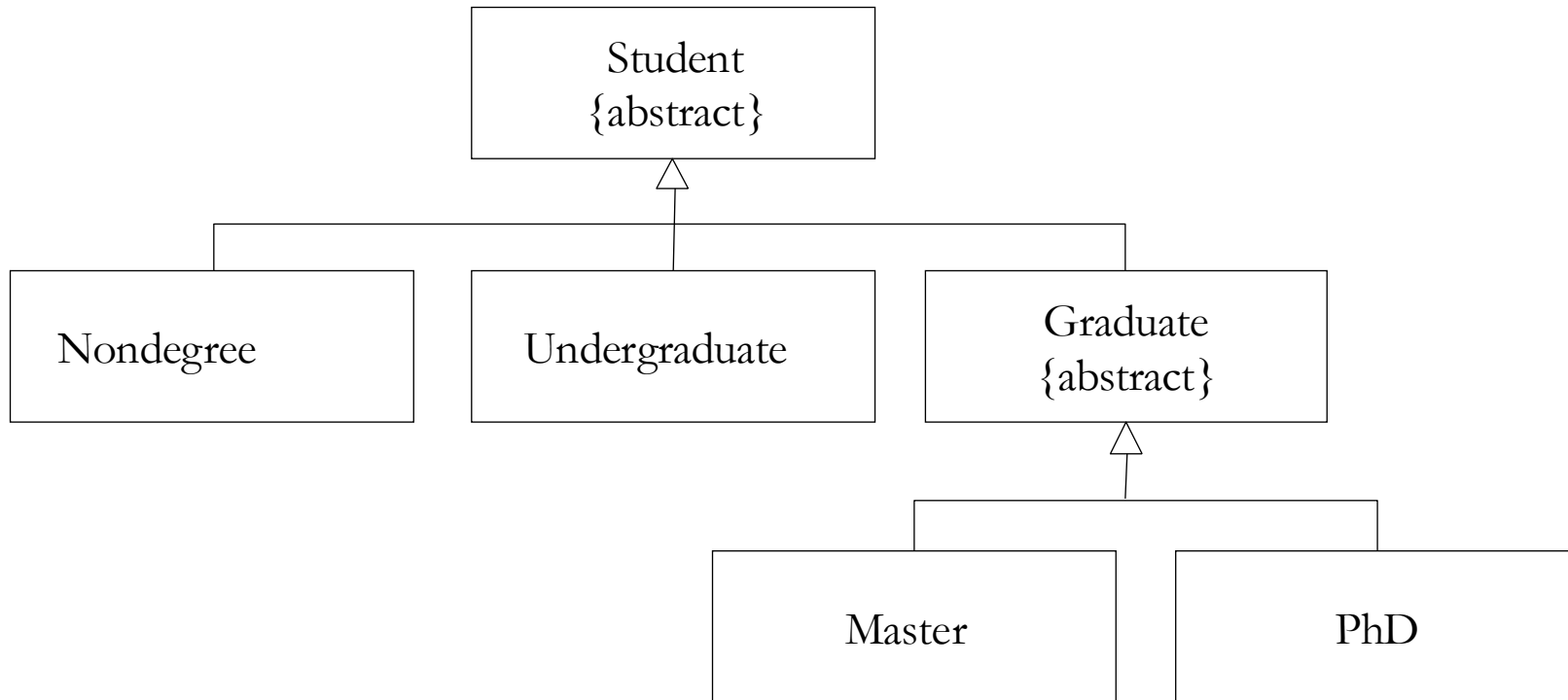
- Important relationship in OO modeling
- Defines a relationship among classes and interfaces.
- Three kinds of inheritances
 - *extension* relation between two classes (*subclasses* and *superclasses*)
 - *extension* relation between two interfaces (*subinterfaces* and *superinterfaces*)
 - *implementation* relation between a class and an interface

Inheritance in UML

- An extension relation is called *specialization* and *generalization*.
- An implementation relation is called *realization*.



Example



Practice Activity

- Draw a UML class diagram showing possible inheritance relationships among classes Person, Employee, and Manager.

Association

- General binary relationships between classes
- Commonly represented as direct or indirect references between classes
- May have an optional label consisting of a name and a direction drawn as a solid arrowhead with no tail.
- The direction arrow indicates the direction of association with respect to the name.



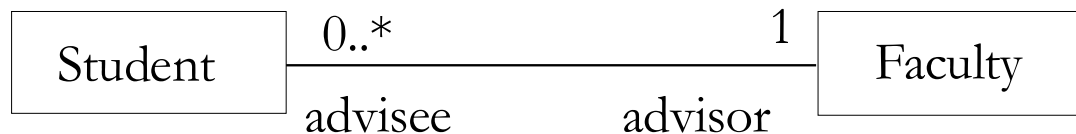
Association (Cont.)

- An arrow may be attached to the end of path to indicate that *navigation* is supported in that direction
- What if omitted?

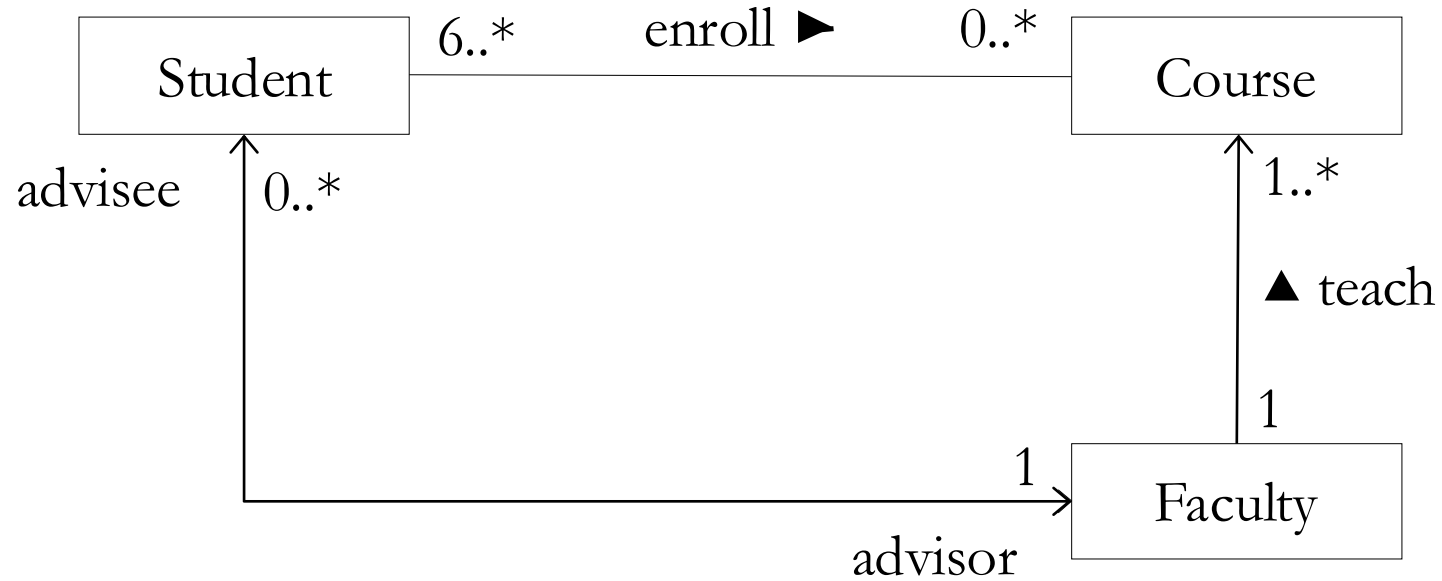


Association (Cont.)

- May have an optional *role name* and an optional *multiplicity specification*.
- The multiplicity specifies an integer interval, e.g.,
 - $m..n$ closed (inclusive) range of integers
 - i singleton range
 - $0..*$ entire nonnegative integer, i.e., 0, 1, 2, ...



Course enrollment example

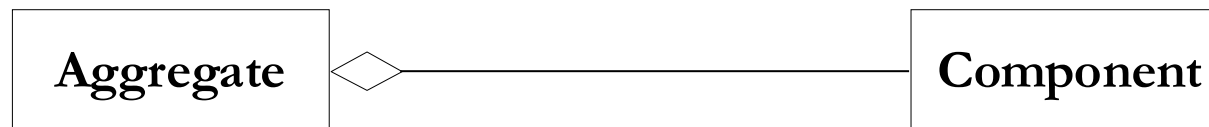


Practice Activity

- Identify possible relationships among the following classes and draw a class diagram
 - Employee
 - Manager
 - Department

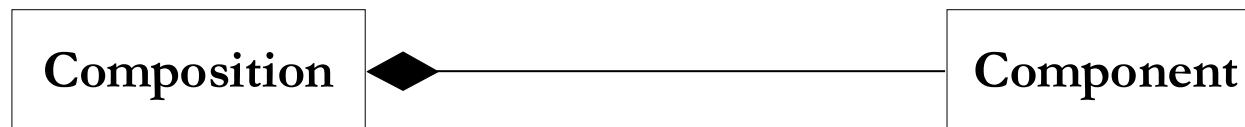
Aggregation

- Special form of association representing *has-a* or *part-whole* relationship.
- Distinguishes the whole (aggregate class) from its parts (component class).
- No relationship in the lifetime of the aggregate and the components (can exist separately).

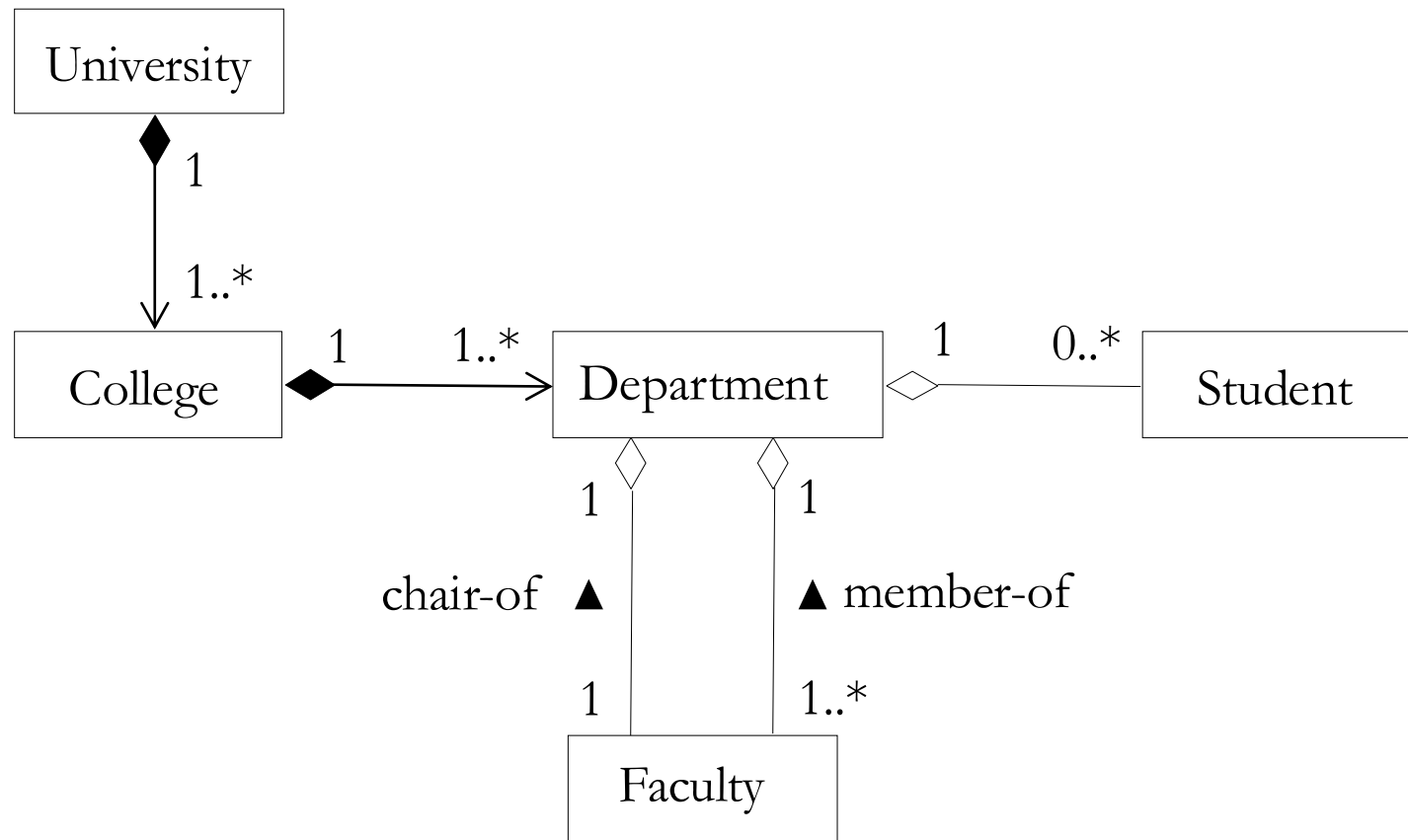


Composition

- Stronger form of aggregation
- Implies exclusive ownership of the component class by the aggregate class
- The lifetime of the components is entirely included in the lifetime of the aggregate (a component can not exist without its aggregate).

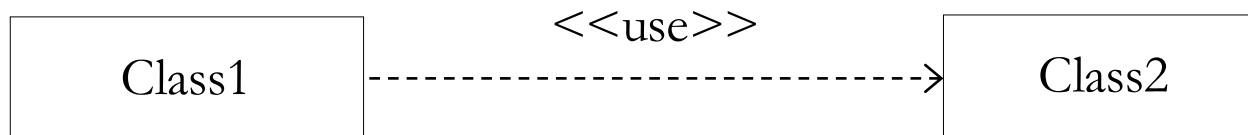


Aggregation Example

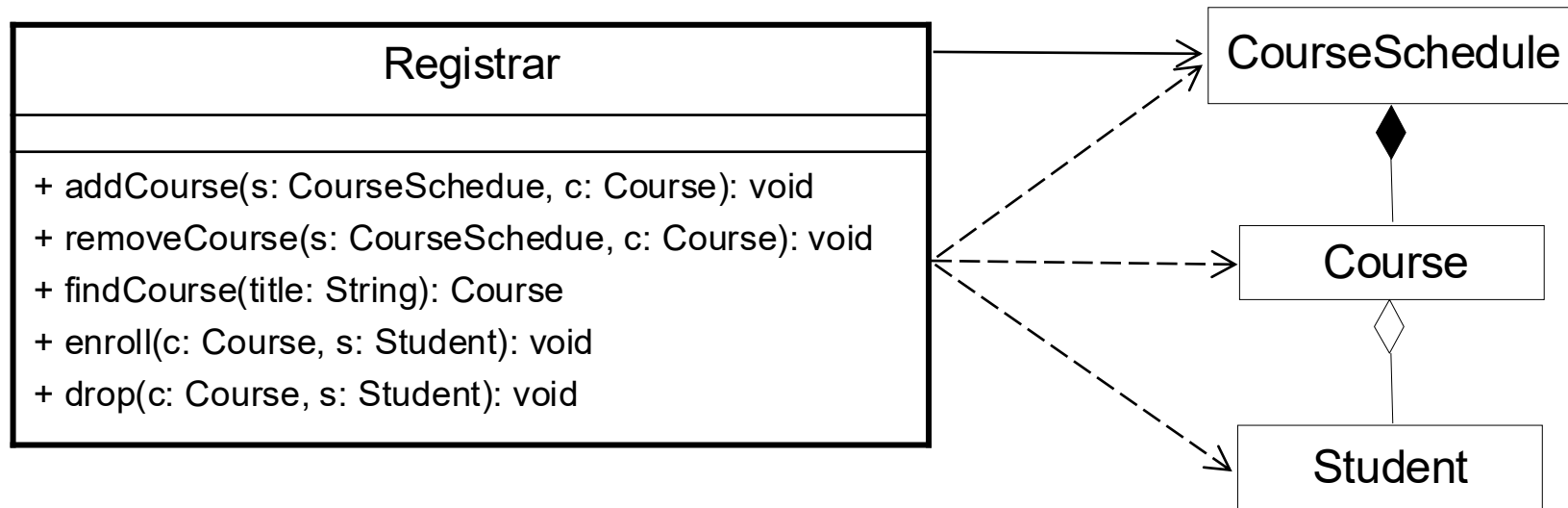


Dependency

- Relationship between the entities such that the proper operation of one entity depends on the presence of the other entity, and changes in one entity would affect the other entity.
- The common form of dependency is the *use* relation among classes.



Dependency Example



Dependencies are most often omitted from the diagram unless they convey some significant information.

Modeling Exercise: E-book Store

Develop an OO model for an e-bookstore. The core requirements of the e-bookstore are to allow its customers to browse and order books, music CDs, and computer software through the Internet. The main functionalities of the system are to provide information about the titles it carries to help customers make purchasing decisions; handle customer registration, order processing, and shipping; and support management of the system, such as adding, deleting, and updating titles and customer information.

1. Identify classes. Classes can represent physical objects, people, organizations places, events, or concepts. Class names should be noun phrases.
2. Identify relevant fields and methods of the classes. Actions are modeled as the methods of classes. Method names should be verb phrases.
3. Identify any inheritance relationships among the classes and draw the class diagram representing inheritance relationships.
4. Identify any association relationships among the classes and draw the class diagram representing association relationships.
5. Identify any aggregation and composition relationships among the classes and draw the class diagram representing dependency relationships.

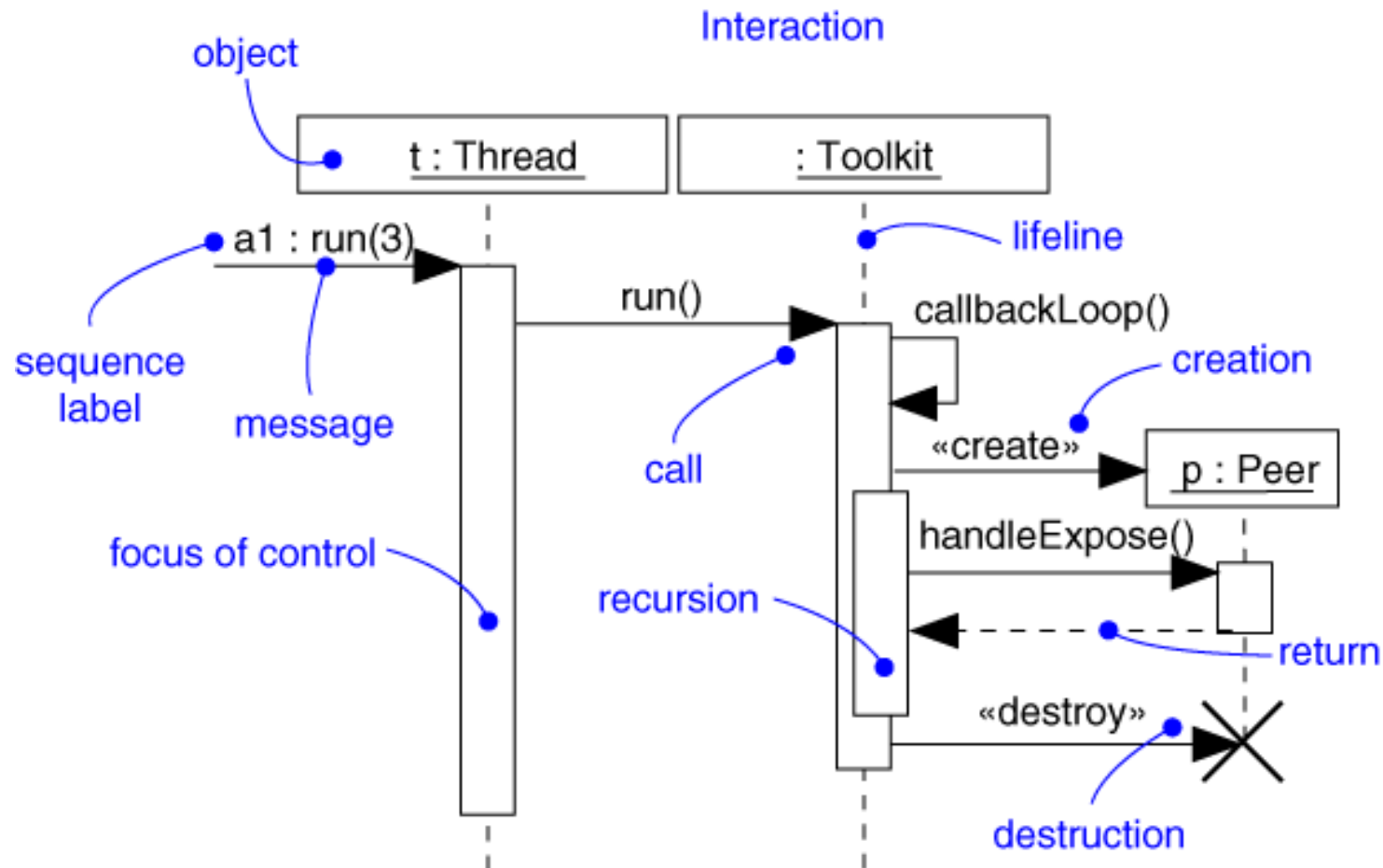
Interaction models

UML Interaction models

- An interaction model shows the interactions that take place between objects in a system
- An interaction “is a behavior that comprises a set of messages exchanged among a set of objects within a context to accomplish a purpose” (UML user guide)
- Interaction models provide a view of system behavior

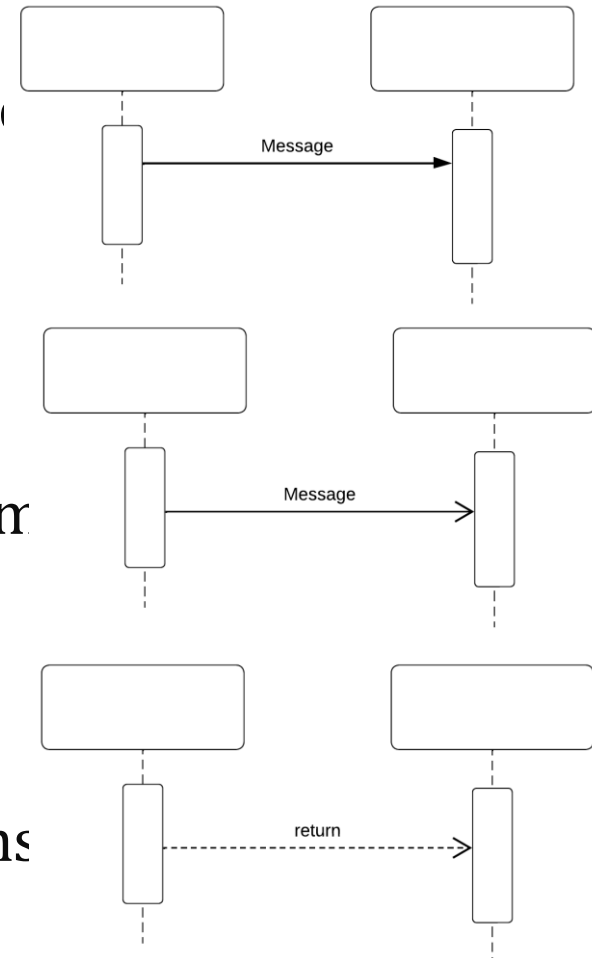
Sequence Diagram

- Captures dynamic behavior (time-oriented)
- Purpose
 - Model flow of control
 - Illustrate typical scenarios



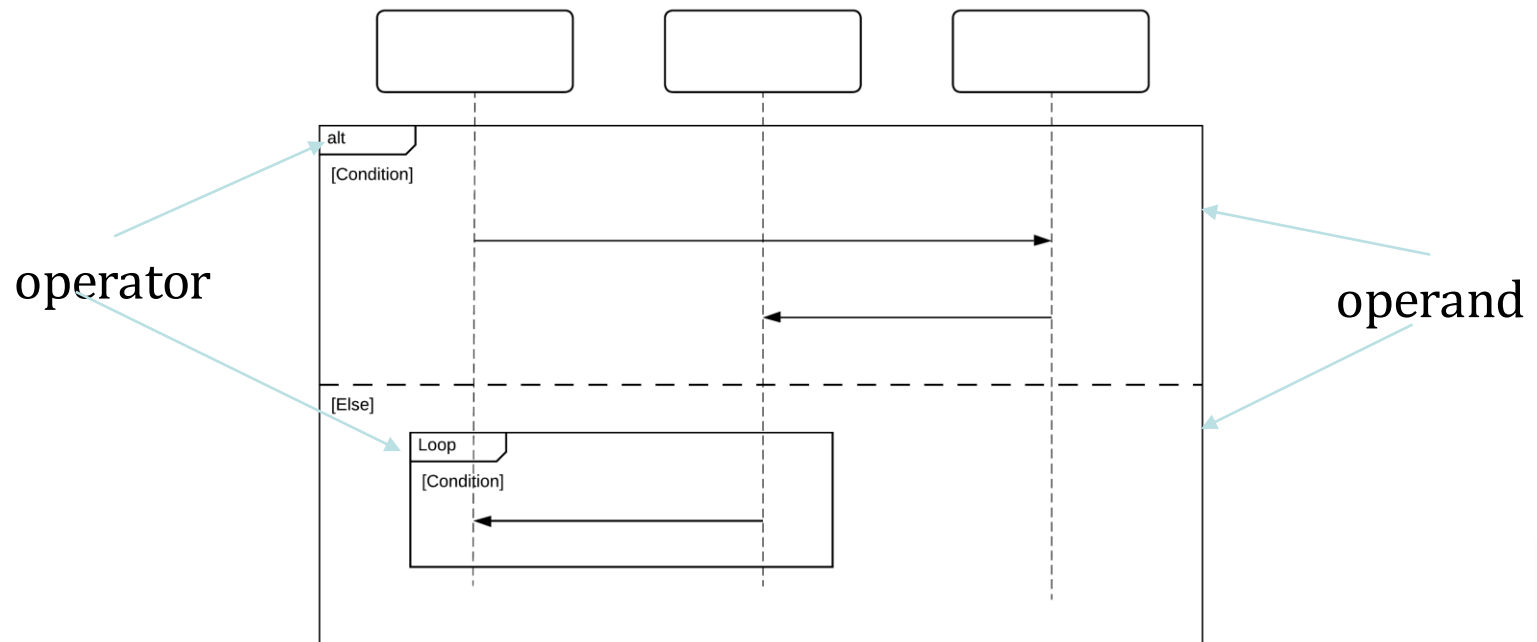
Sequence Diagram – Main Message types

- Synchronous Message
 - Sender waits till the return message is received before next
- Asynchronous Messages
 - Sender does not wait for response message
- Response message
 - Not mandatory in obvious situations



Sequence Diagram – Combined Fragments

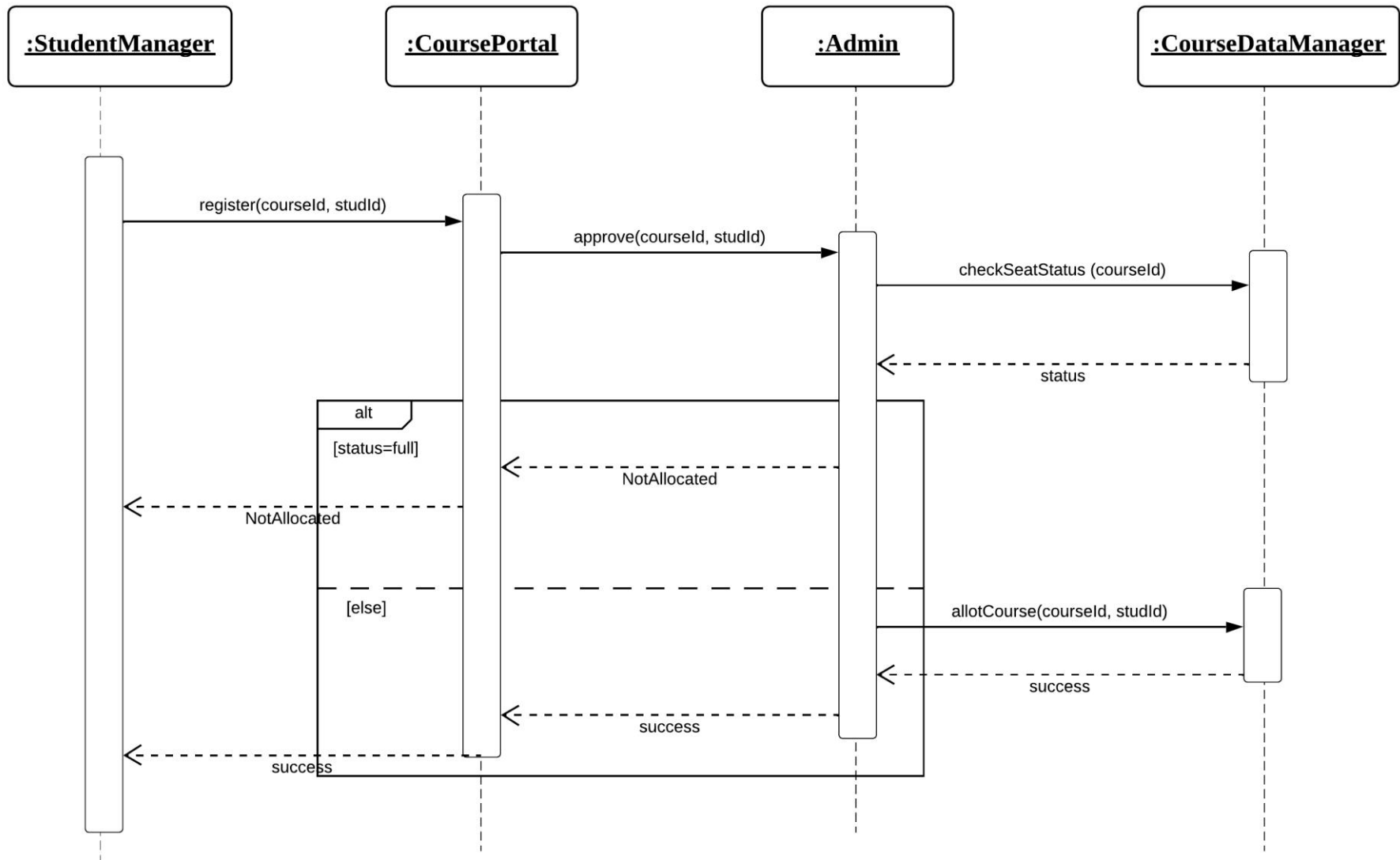
- Model control structures explicitly
- UML sequence diagram supports 12 operators. Three groups
 - Branches and loops, Concurrency and order, Filters and Assertions



Different Operators

Name and Operator	Use
Alternative (alt)	Express alternative execution (if-else)
Optional (opt)	Fragment executes based on guard condition (if)
Break (break)	Execution of a fragment when break condition is met
Loop (loop)	Repeated execution of a fragment
Sequential (seq)	Weak ordering (default model)
Strict (strict)	Interaction with strict order
Parallel (par)	Concurrent execution of sub-scenarios
Critical (critical)	Atomic interactions (no other interactions can affect)
Ignore (ignore)	Irrelevant messages (insignificant messages at runtime)
Consider (consider)	Important messages of the interactions
Negate (neg)	Model invalid interactions, undesirable situations
Assert (assert)	To assert certain interactions (mandatory)

Sequence Diagram – Example



Interaction Modeling Tips

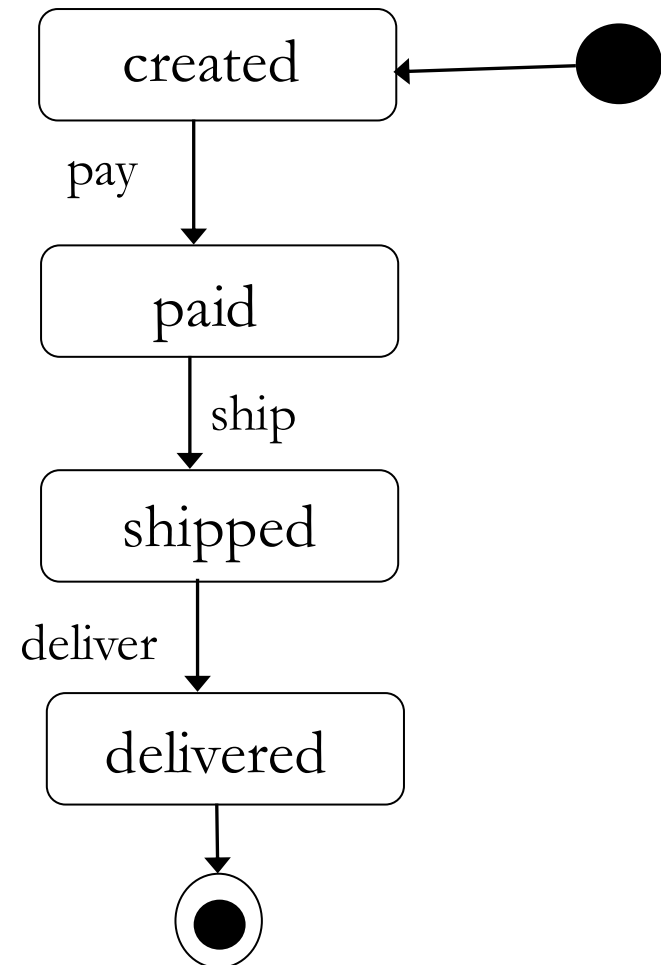
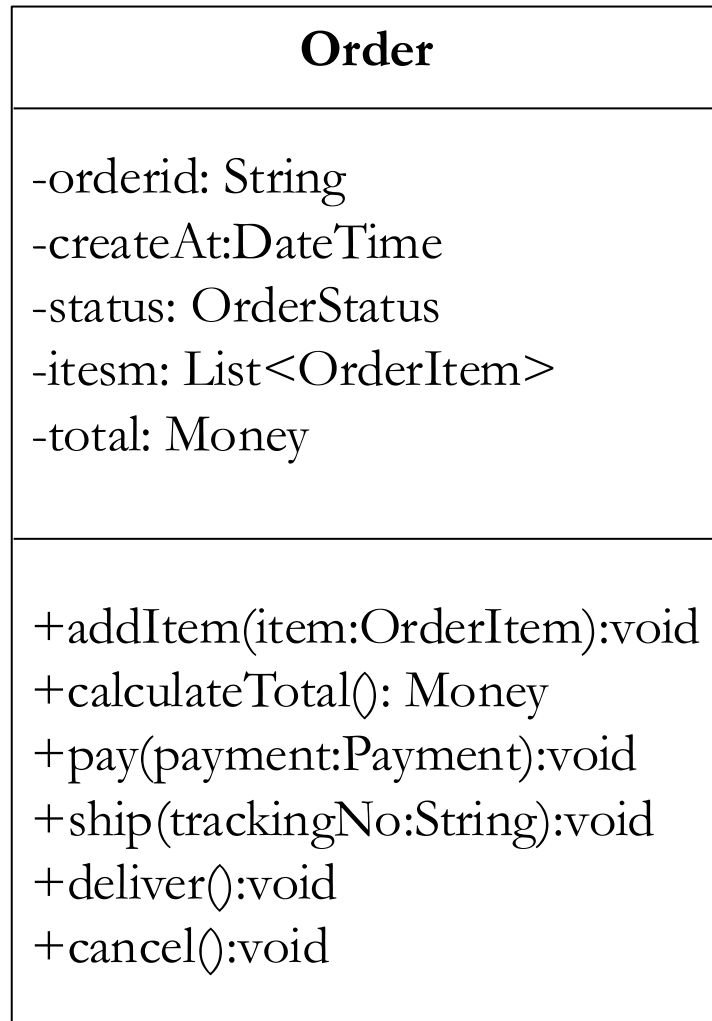
- Set the context for the interaction.
- Express the flow from left to right and from top to bottom.
- Put active objects to the left/top and passive ones to the right/bottom.
- Use sequence diagrams
 - to show the explicit ordering between the stimuli
 - when modeling real-time

Understanding How Systems Act Over Time

SDLC → Planning → Requirements → Modeling → UML (Class + Sequence) → **Behavioral Modeling (using State Diagrams)**

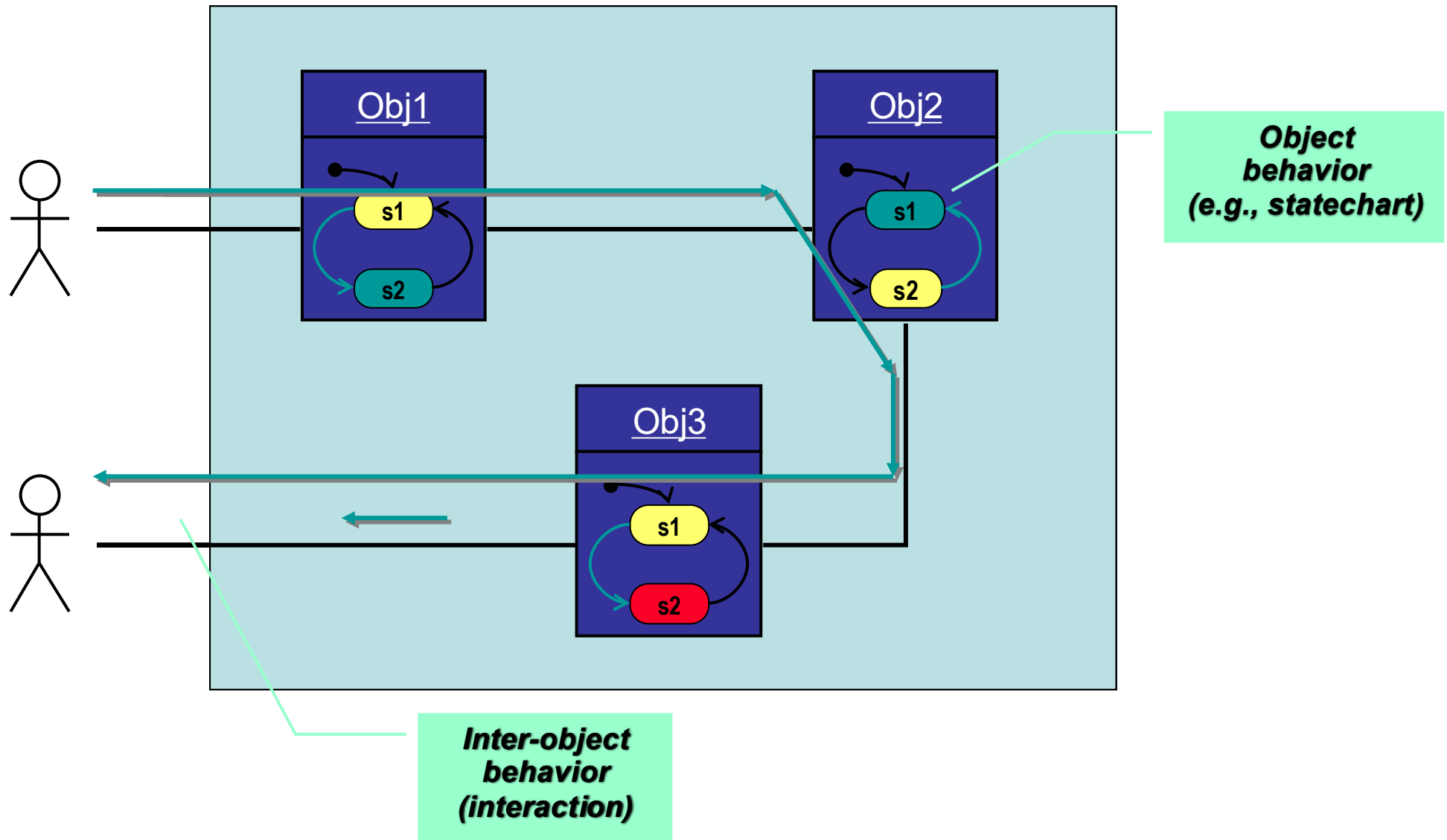
“Let’s begin by asking: why do we even need behavioral models when we already have structure and interactions captured?”

Why structural & interaction models are not enough?

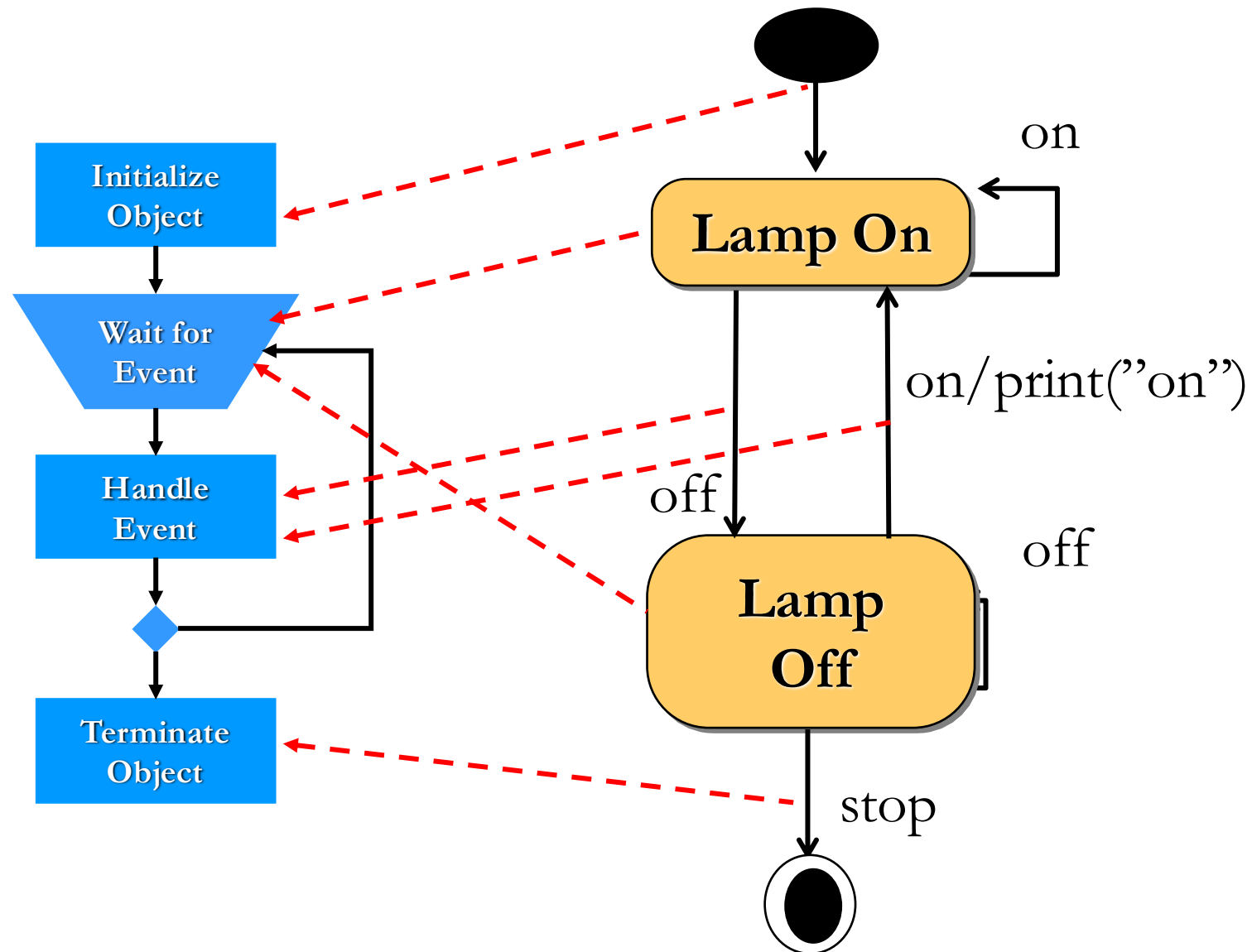


Specifying interactions/behavior using UML

- In UML, all behavior results from the actions of (active) objects



Object Behavior and State diagram



Reflecting object lifecycle in a state diagram

State diagrams

- A *state diagram* specifies the life histories of objects in terms of the sequences of operations that can occur in response to external stimuli.
 - For example, a state diagram can describe how an object responds to a request to invoke one of its methods.
- A state diagram describes behavior in terms of sequences of *states* that an object can go through in response to *events*.

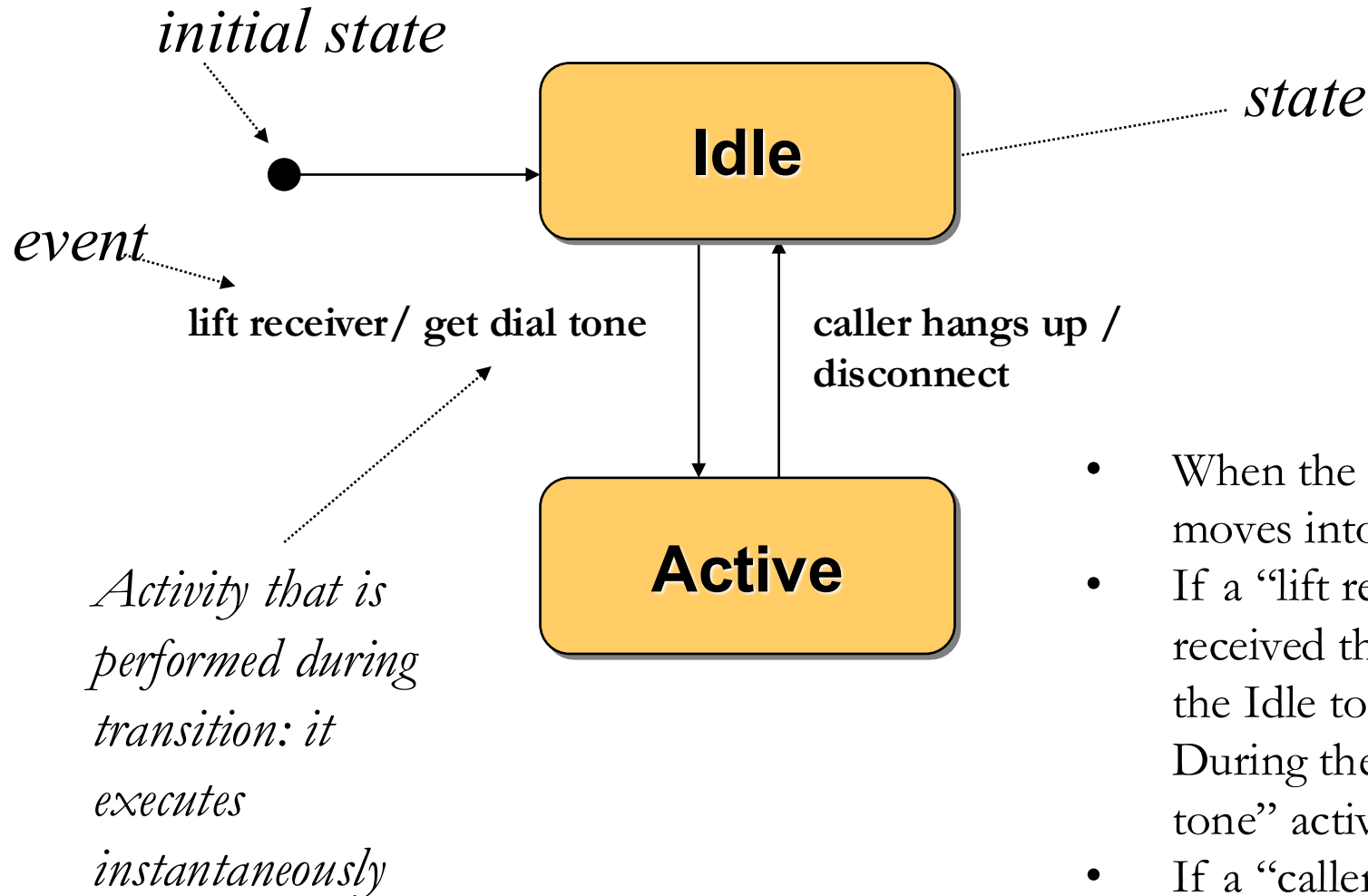
Key Concepts

- An *event* is a significant or noteworthy occurrence at a point in time.
 - Examples of events: sending a request to invoke a method, termination of an activity.
 - An event occurs instantaneously in the time scale of an application.
- A *state* is a condition of an object during its lifetime.
 - For example, a student is in the registered state after completing course registration.
 - A state is an abstraction of an object's attribute values and links
 - For example, a bank account is in the Overdraft state when the value of its balance attribute is less than 0.

Key Concepts - 2

- A *transition* occurs when an event causes an object to change from its current (source) state to a target state.
 - For example, if a student is in the registered state and then drops out of the program then the student is in the “not registered” state.
 - The source and target states can be the same.
 - A transition is said to fire when the change from source to target state occurs.
- A *guard condition* on a transition is a boolean expression that must be true for a transition to fire
- An *activity* is a behavior that is executed in response to an event.

Simple Example: Telephone Object



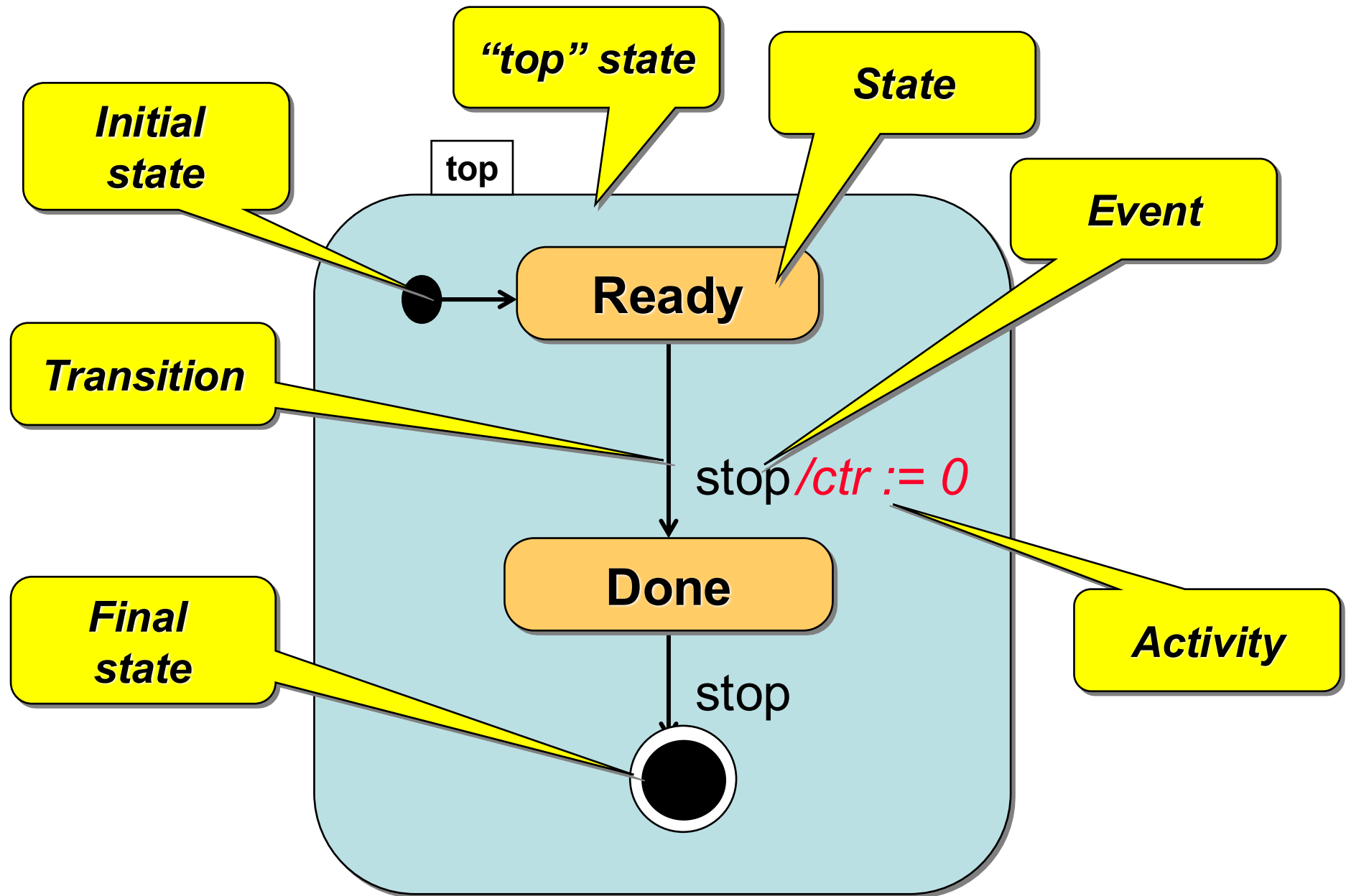
- When the object is created, it moves into the Idle state.
- If a “lift receiver” event is received the object moves from the Idle to the Active state. During the transition the “get dial tone” activity is executed.
- If a “caller hangs up” event occurs when in Active state, the object moves to the Idle state and the “disconnect” activity is performed during the transition.

Simple Transitions

- A transition is a relationship between two object states indicating that an object will move from one state to the other and perform certain actions when a specified event occurs.

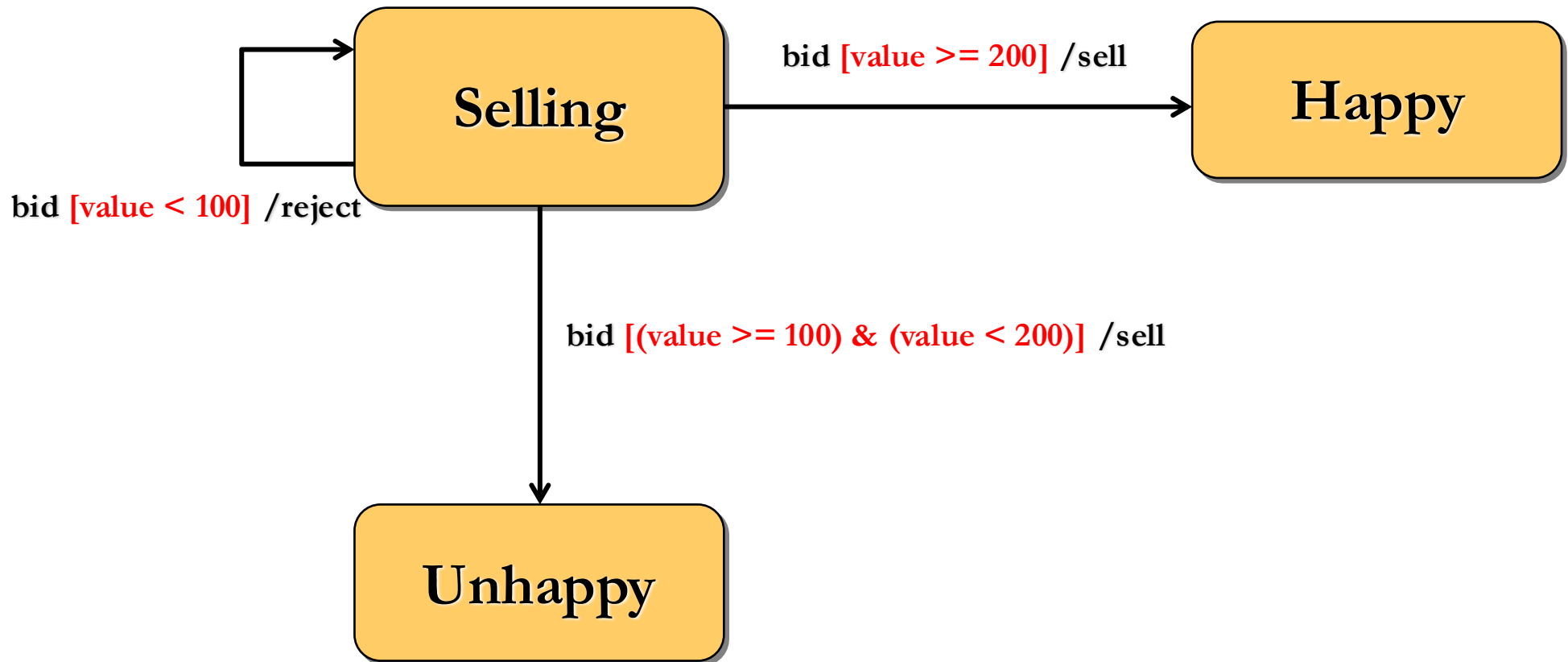
Event(args)[condition]/activity^destination.message(params)

Basic UML State Diagram



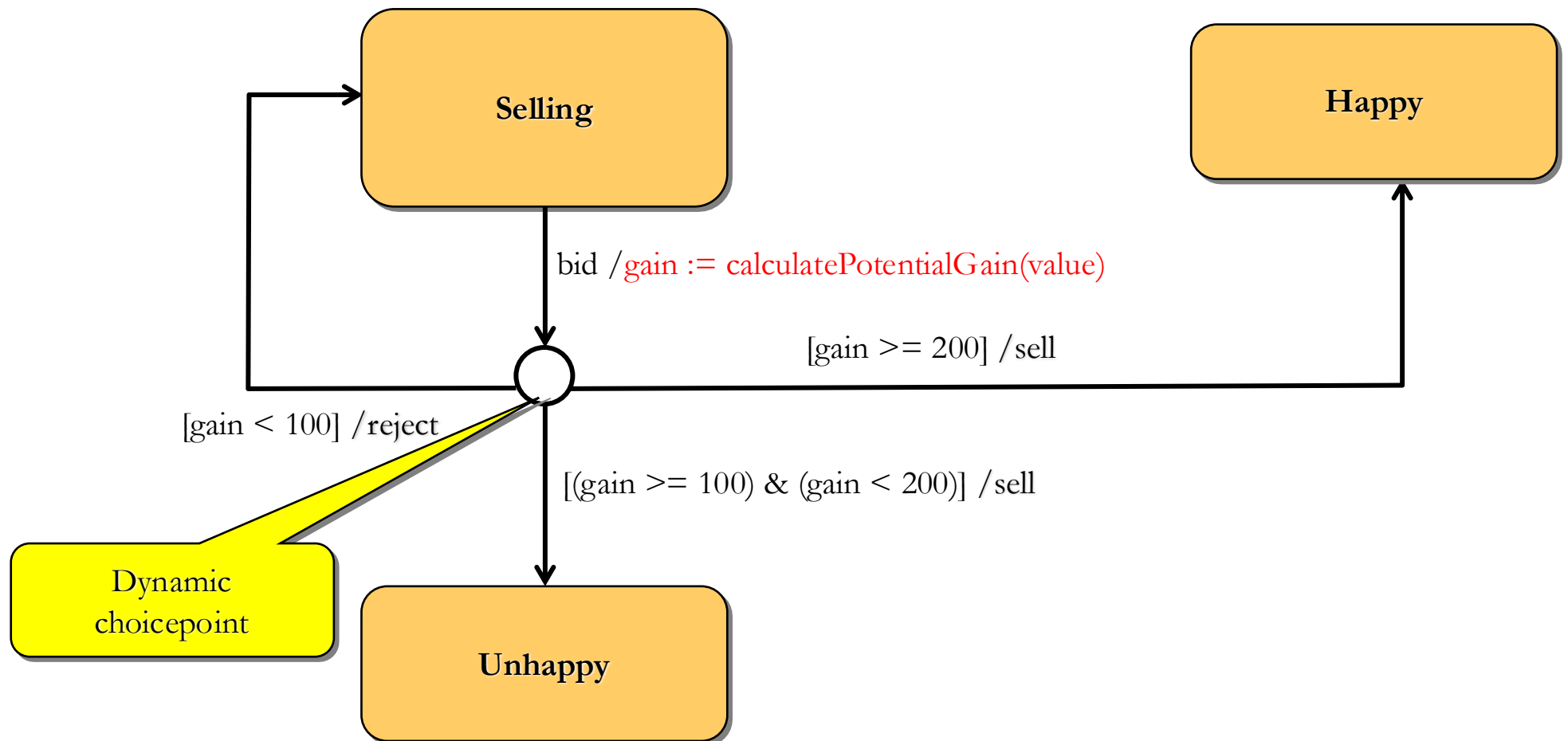
Guards

- Conditional execution of transitions
 - guards (Boolean predicates) must be side-effect free

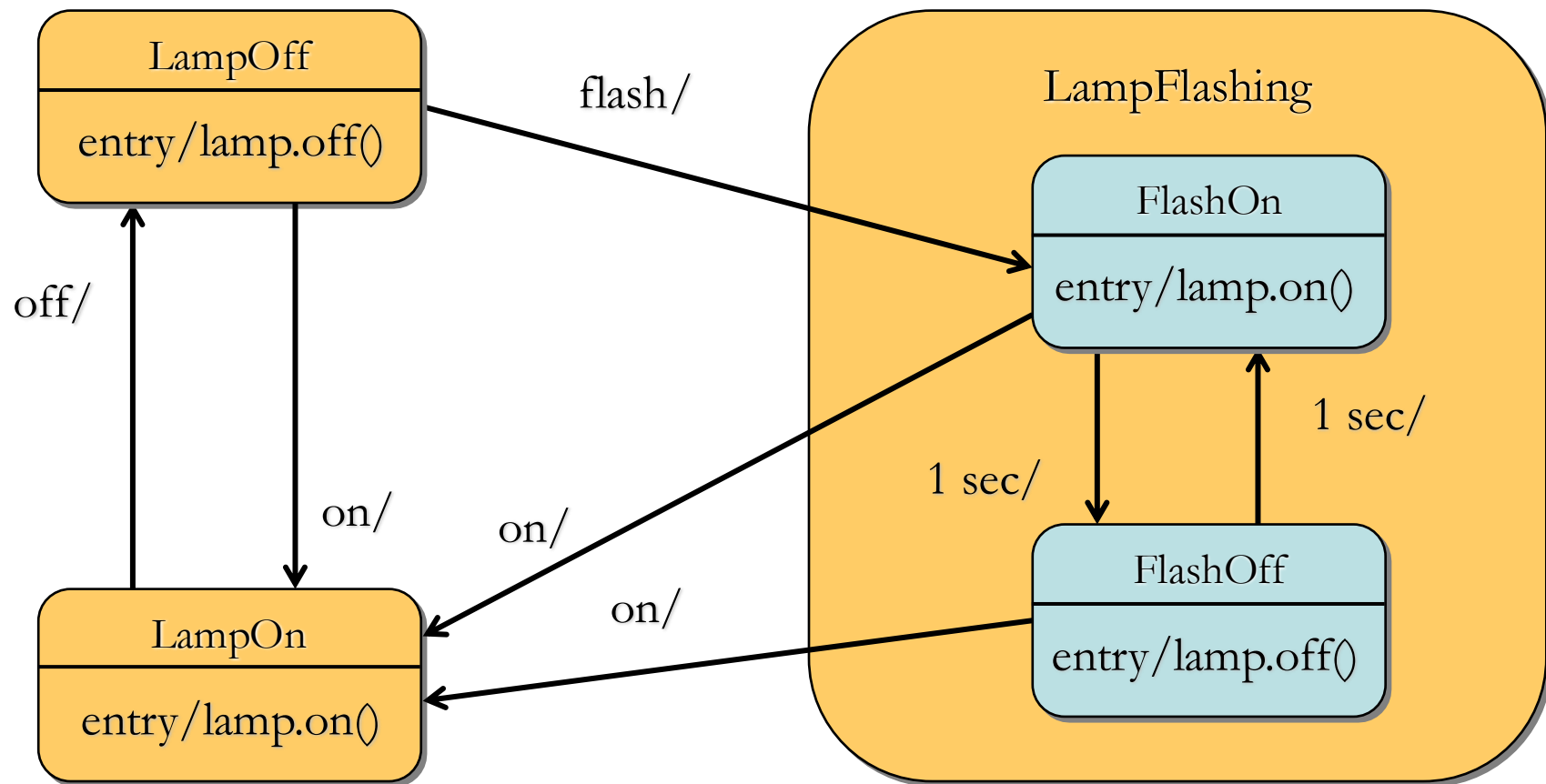


Dynamic Conditional Branching

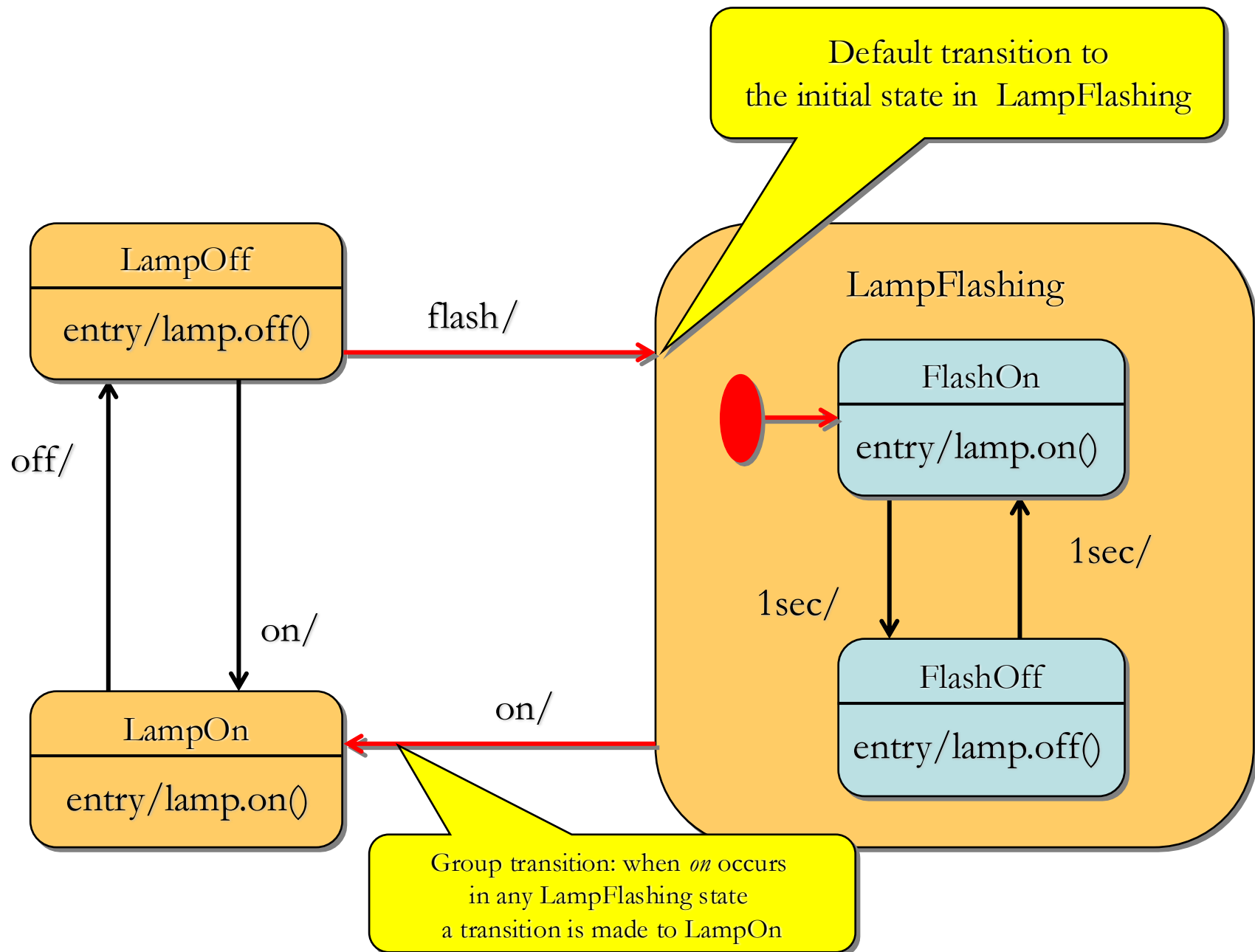
Choice pseudostate: guards are evaluated at the instant when the decision point is reached



Hierarchical State Machines

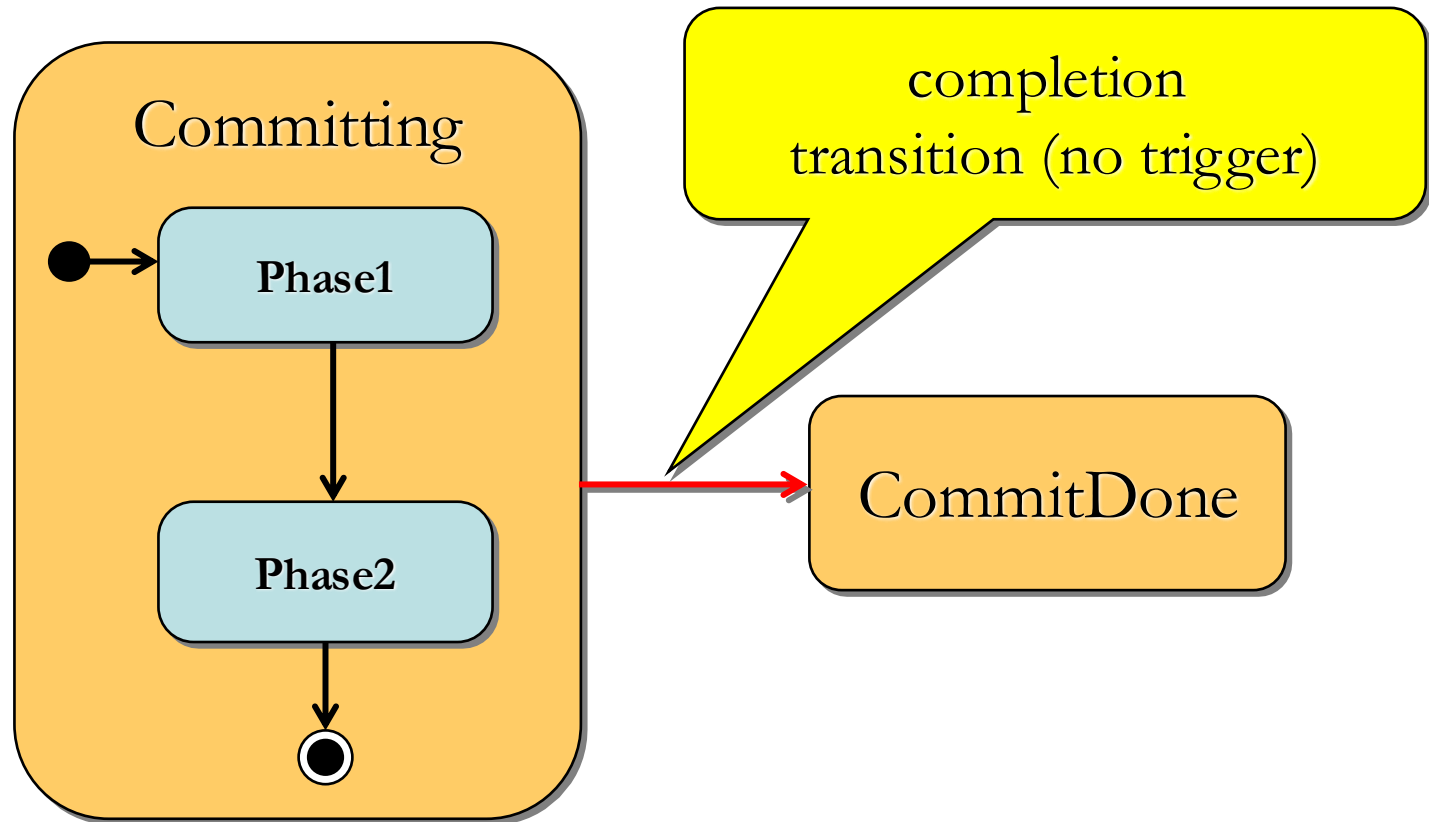


Group Transitions



Completion Transitions

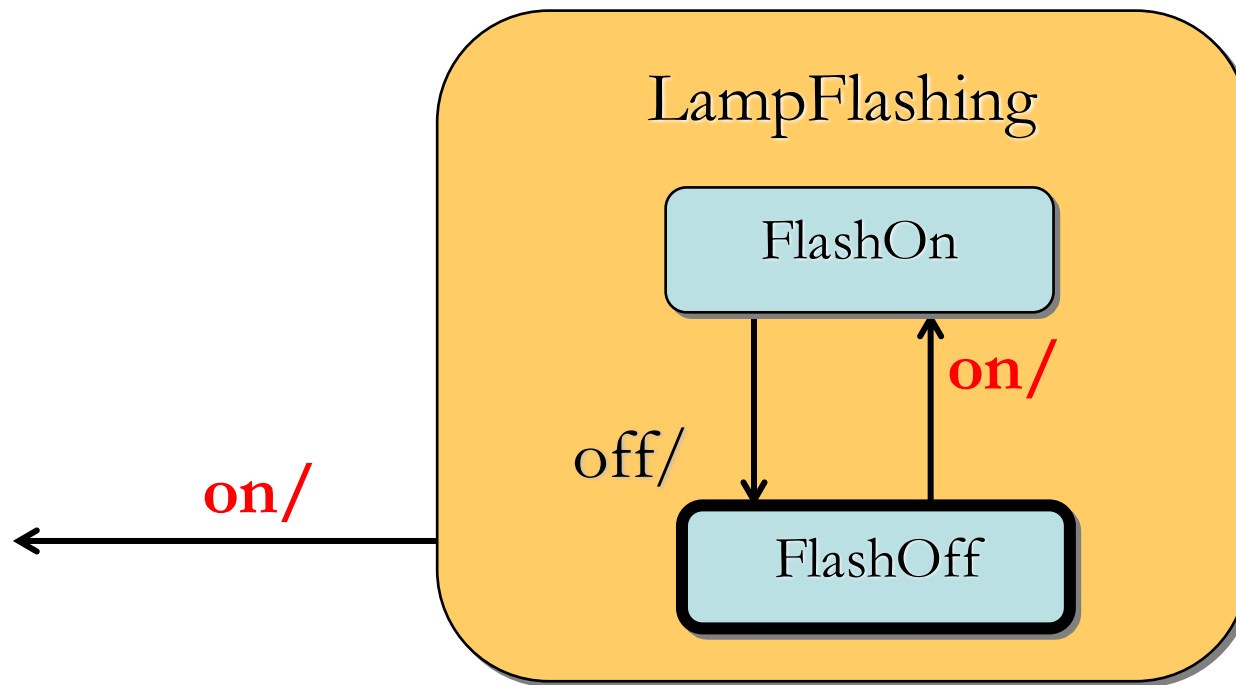
- Triggered by a completion event
 - generated automatically when an immediately nested state machine terminates



Triggering Rules

Two or more transitions may have the same event trigger

- innermost transition takes precedence
- event is discarded whether or not it triggers a transition



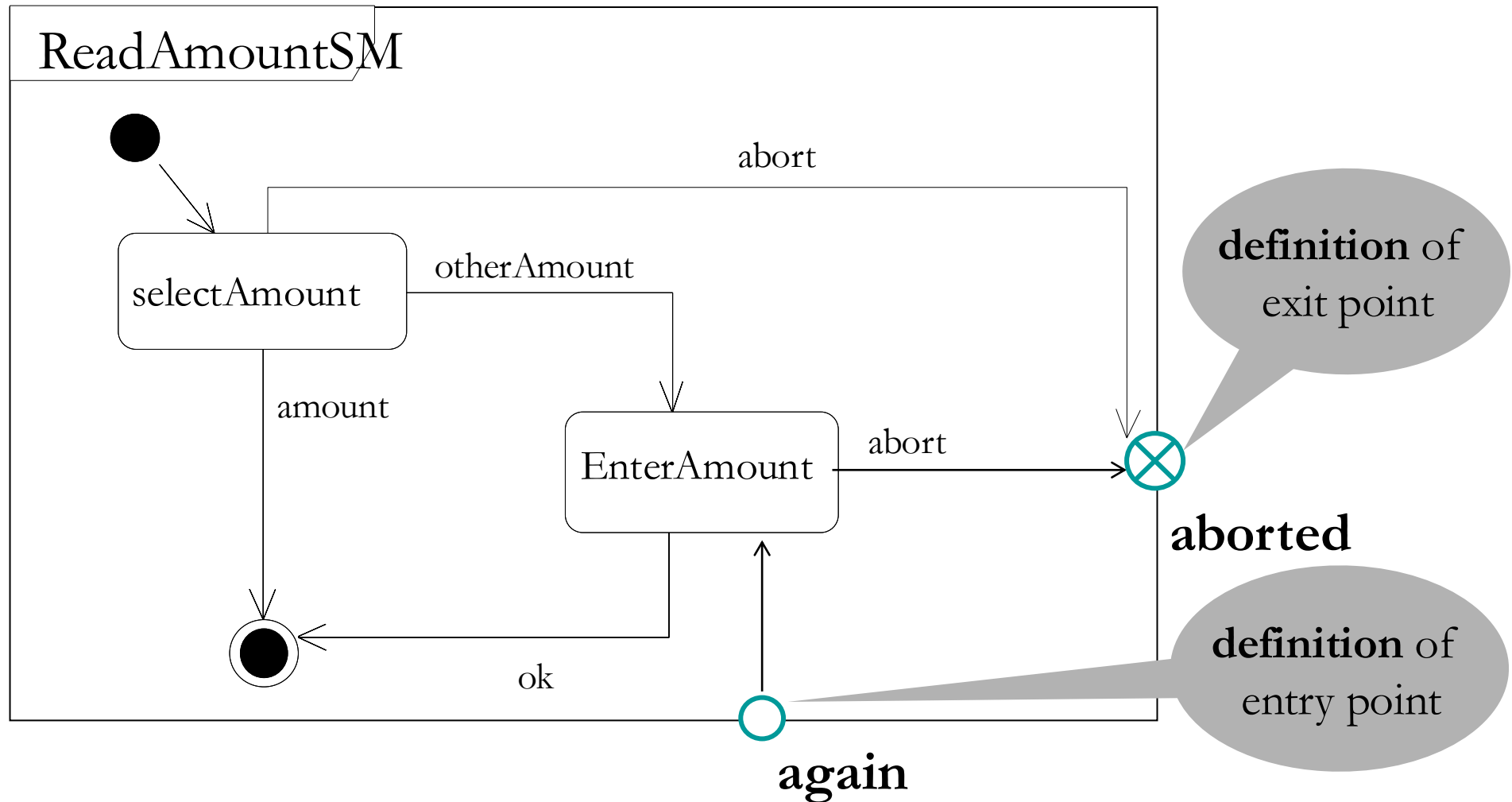
State Syntax - Submachines



- *State name:submachine*
 - machine is the name of a (nested) state machine that has an initial and final state.
 - Execution of the nested machine begins in the initial state.
 - When final state of the nested machine is reached then the exit action of the parent state is executed (if it exists).

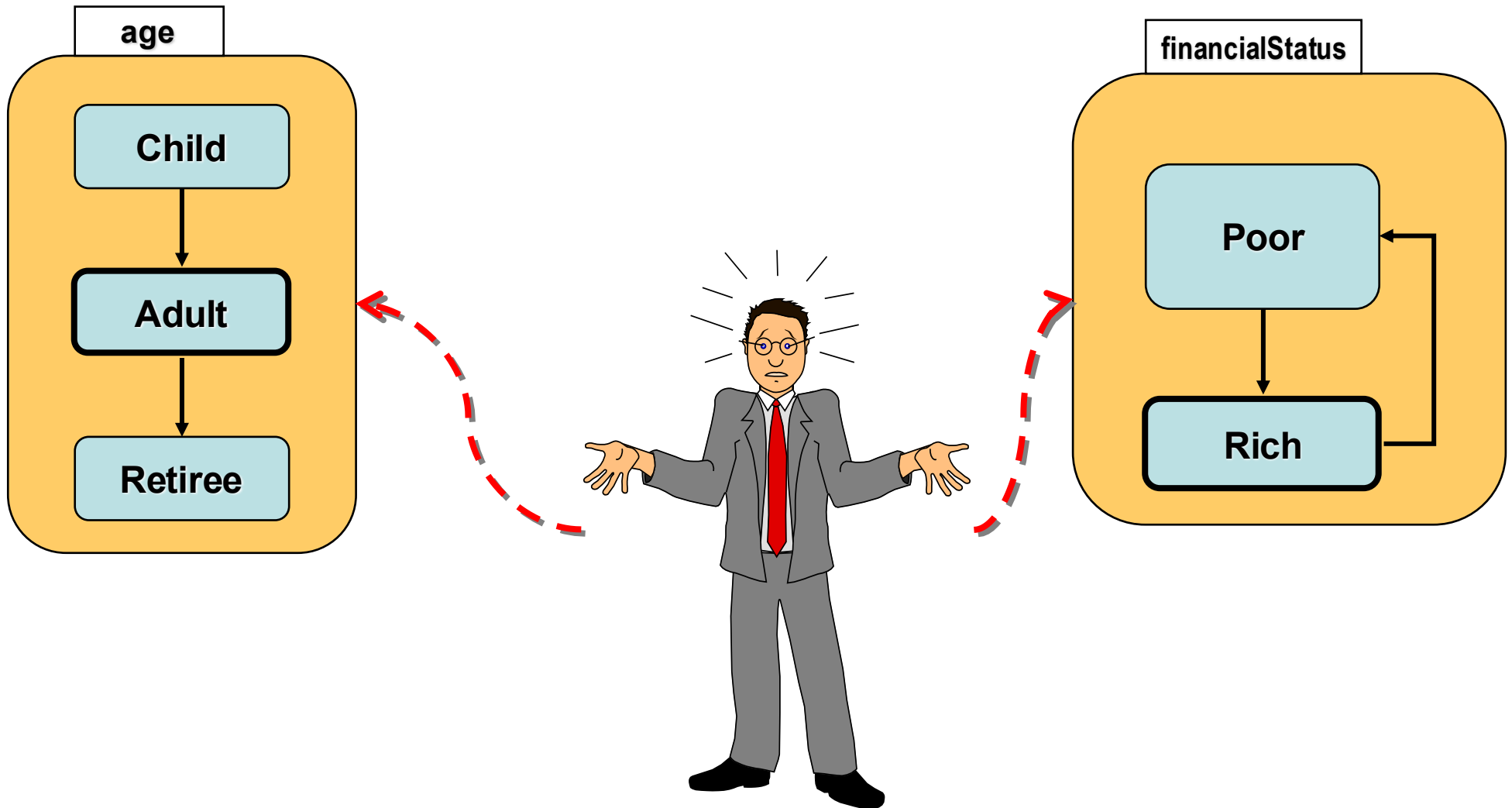
UML 2.0: Entry/Exit Points

Encapsulation of submachines



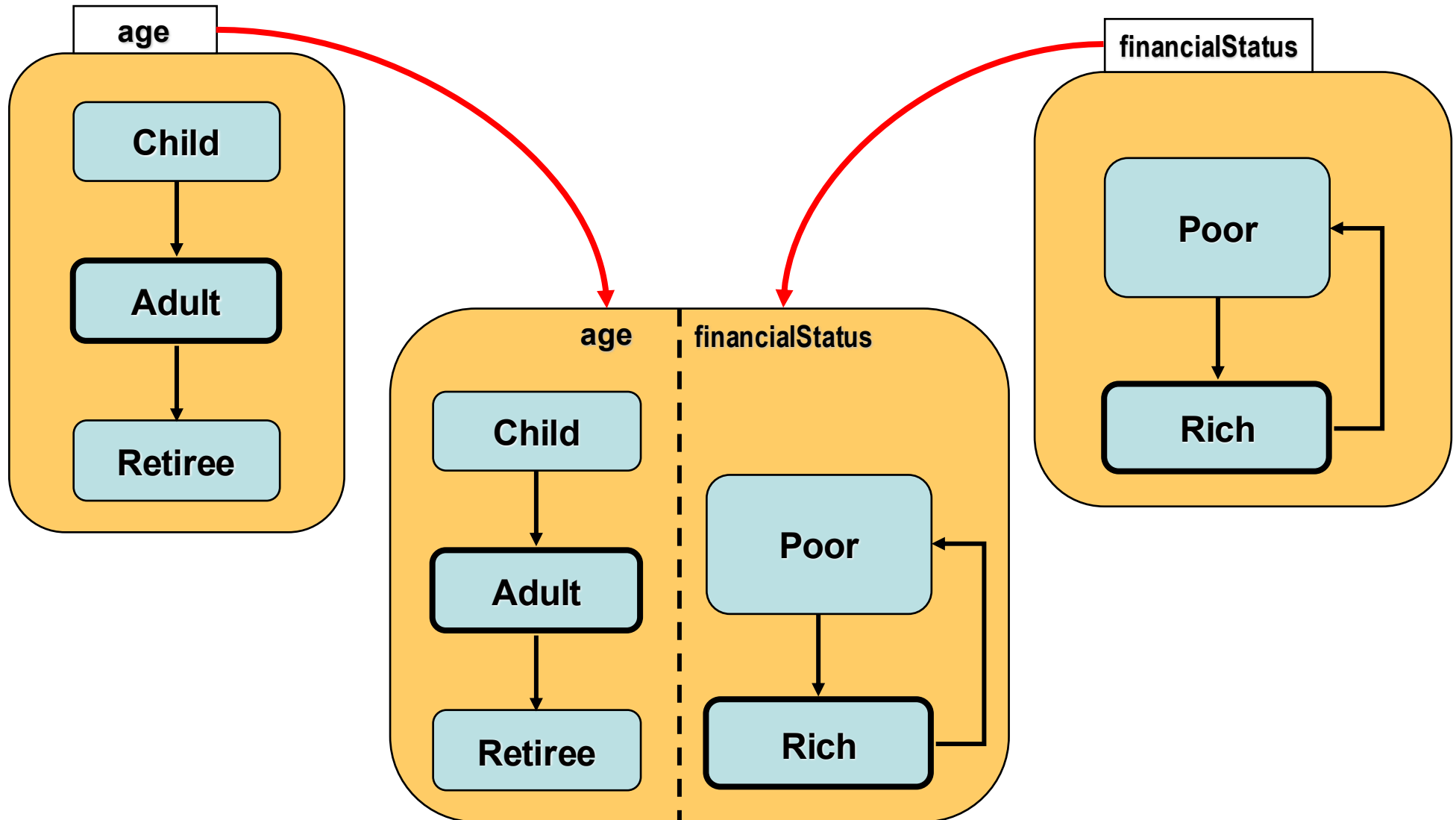
Orthogonality

Multiple simultaneous perspectives on the same entity



Orthogonal Regions

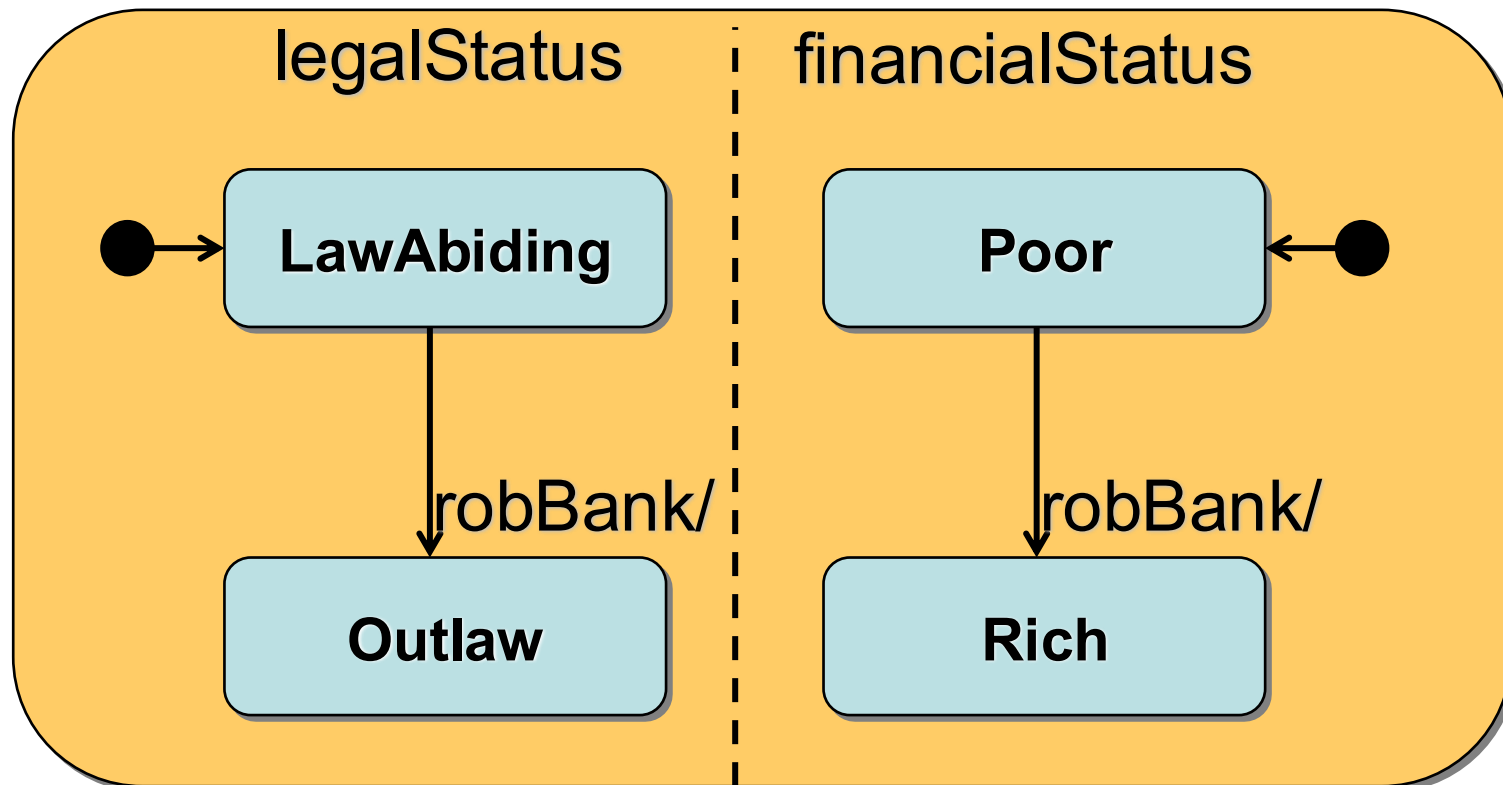
Combine multiple simultaneous descriptions



Orthogonal Regions - Semantics

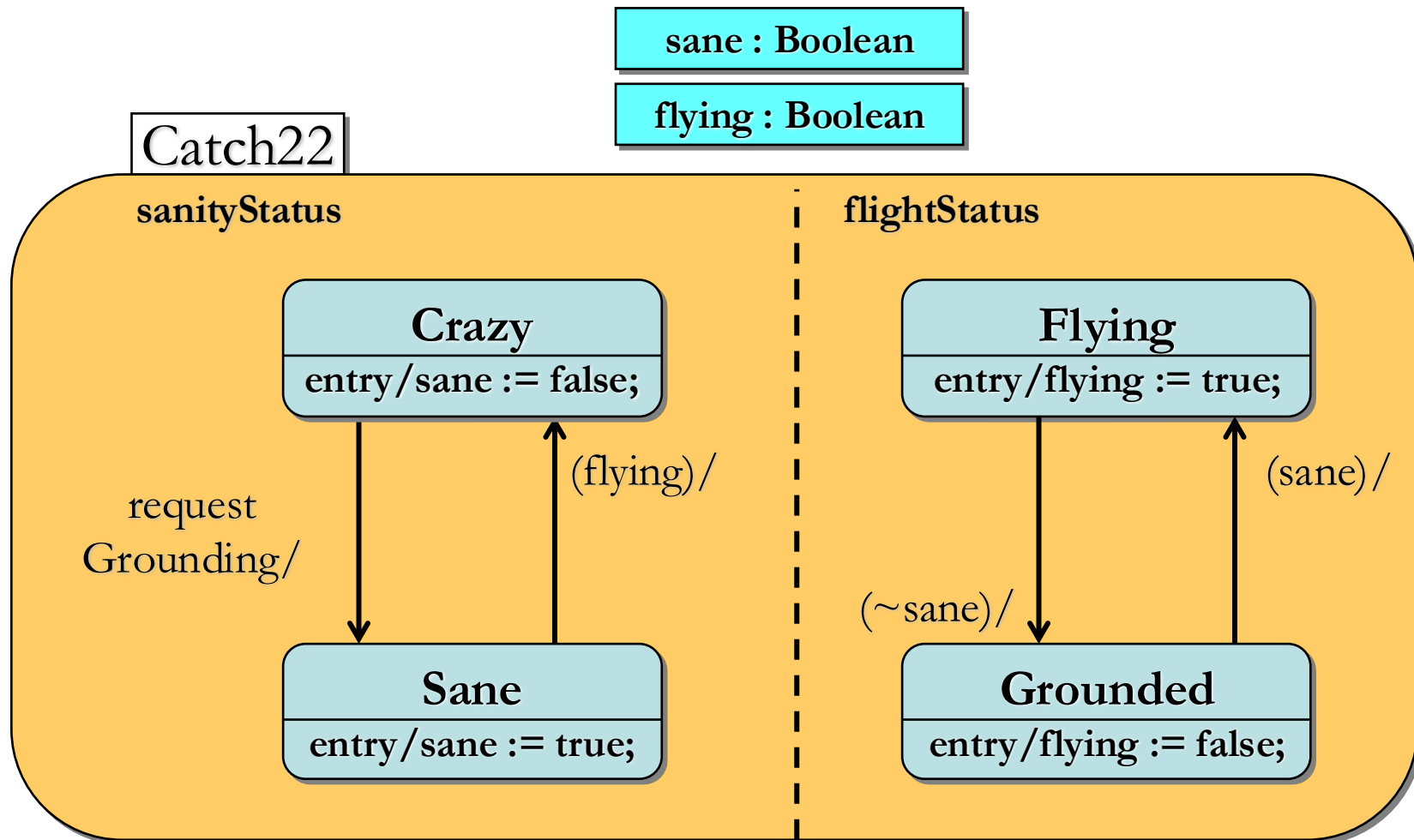
All mutually orthogonal regions detect the same events and respond to them “simultaneously”

- usually reduces to interleaving of some kind



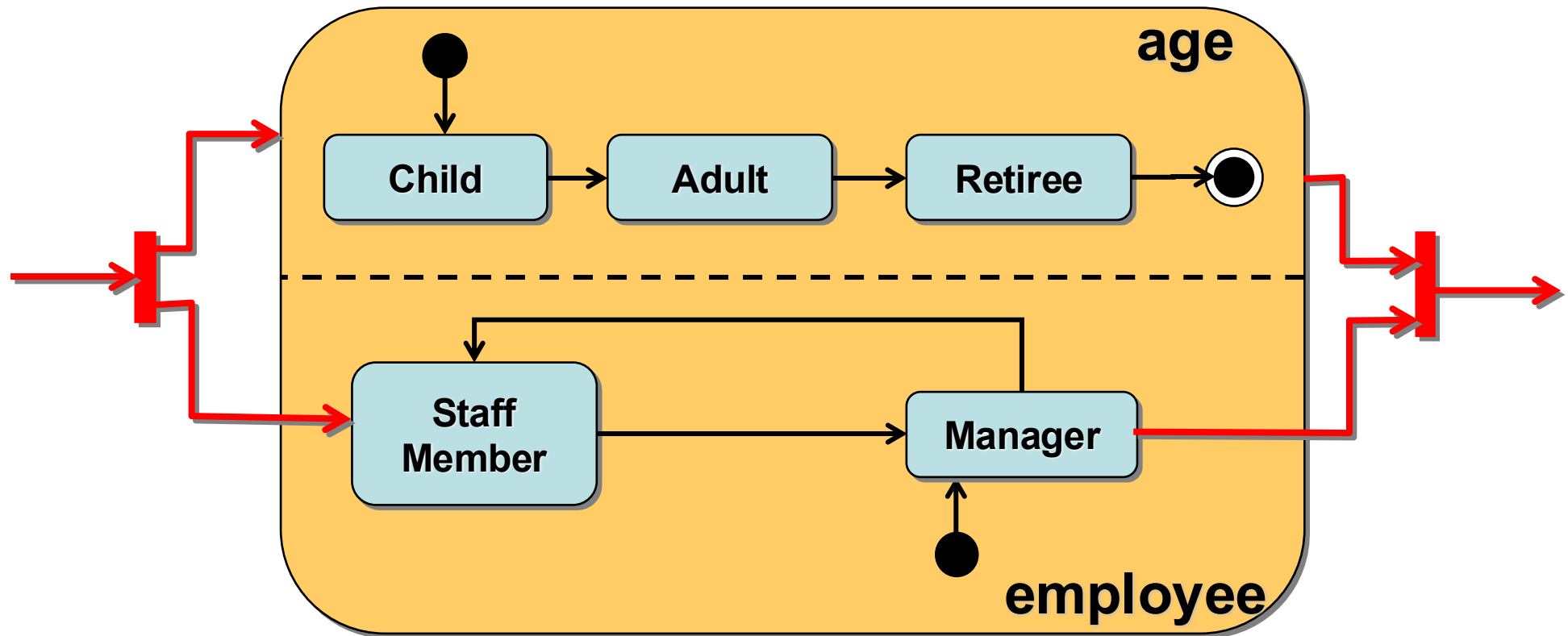
Interactions Between Regions

Typically through shared variables or awareness of other regions' state changes

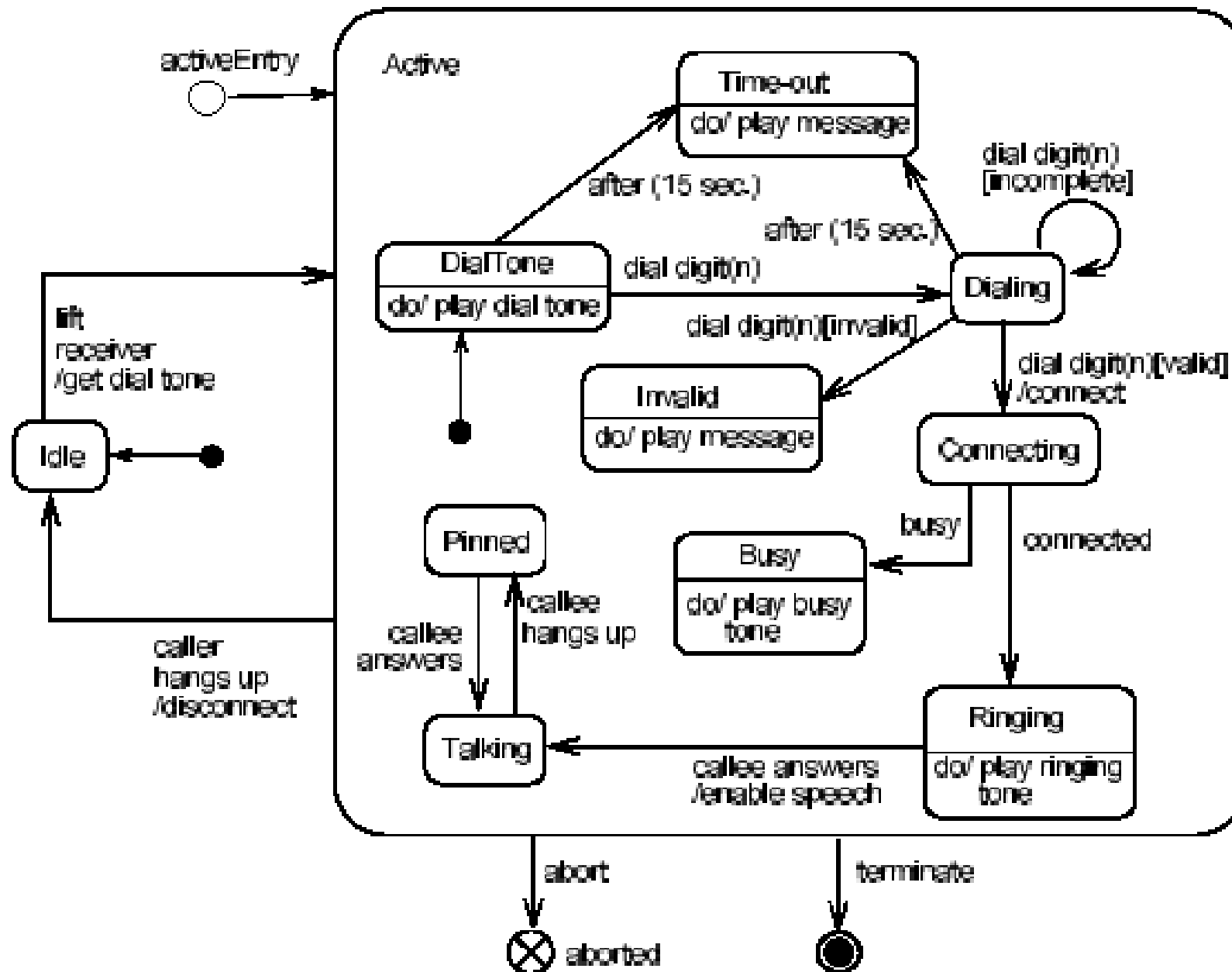


Transition Forks and Joins

For transitions into/out of orthogonal regions



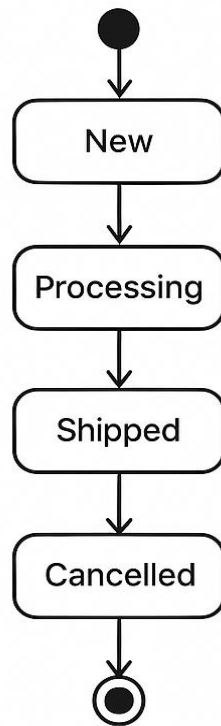
Telephone state machine - Example



From Object Behavior to System workflows

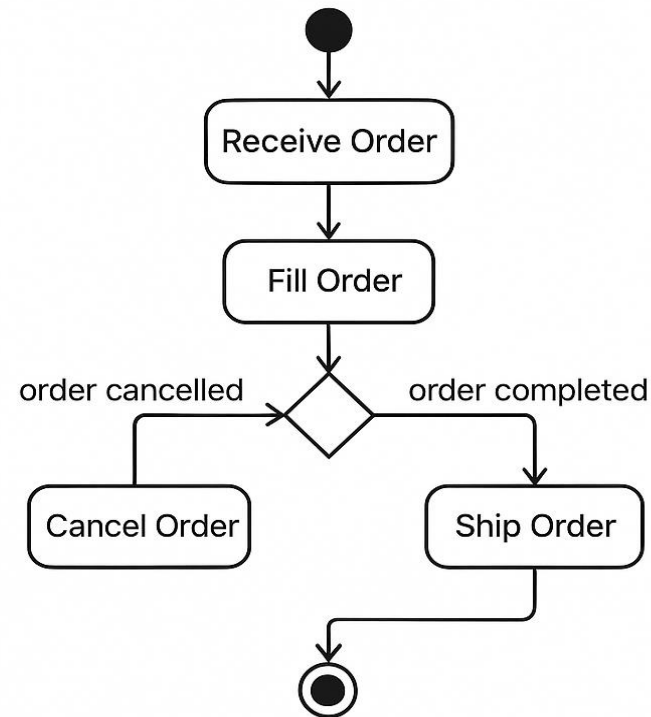
State Machine

Order states



Activity

Order Processing workflow



- State machines tell us what one object does. But in real systems, several entities cooperate in a process — ordering, payment, notification.
- Activity diagrams help us visualize such workflows

Activity Diagrams

- UML Activity diagrams can be used to show sequential and parallel activities in a process. They are useful for modeling business processes, workflows, data flows, and complex algorithms
 - The diagram can show both *control* flow and *data* flow

Core Elements of an Activity Diagram

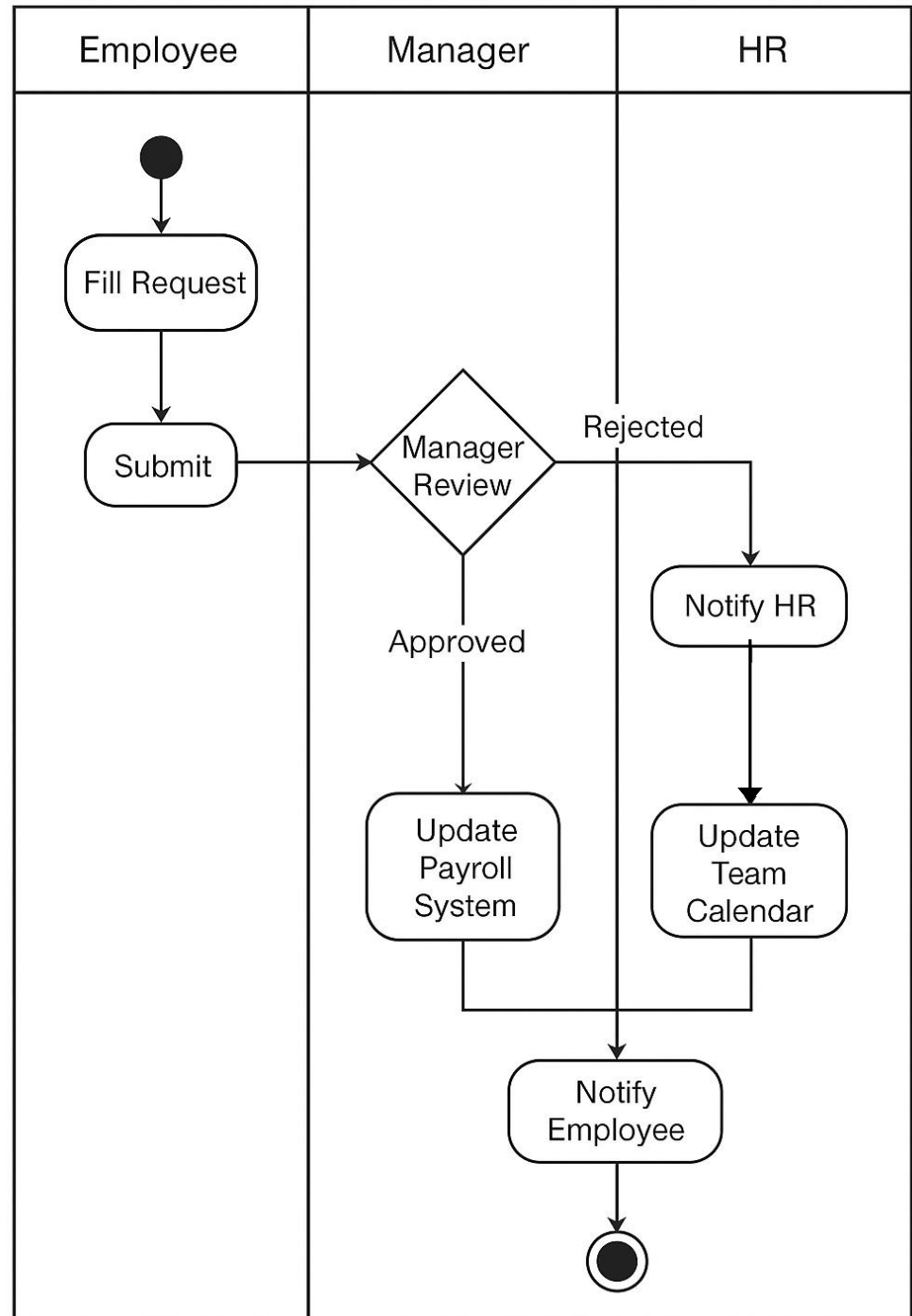
Symbol	Meaning	Example
●	Initial Node	Workflow starts when request is received
◇	Decision / Merge	Payment successful?
⊥	Fork / Join	Parallel branches (e.g., send email & update database)
Rounded Rectangle	Action / Activity	“Validate Login”, “Process Payment”
Arrow	Control Flow	Order of execution

How to apply Activity Diagrams?

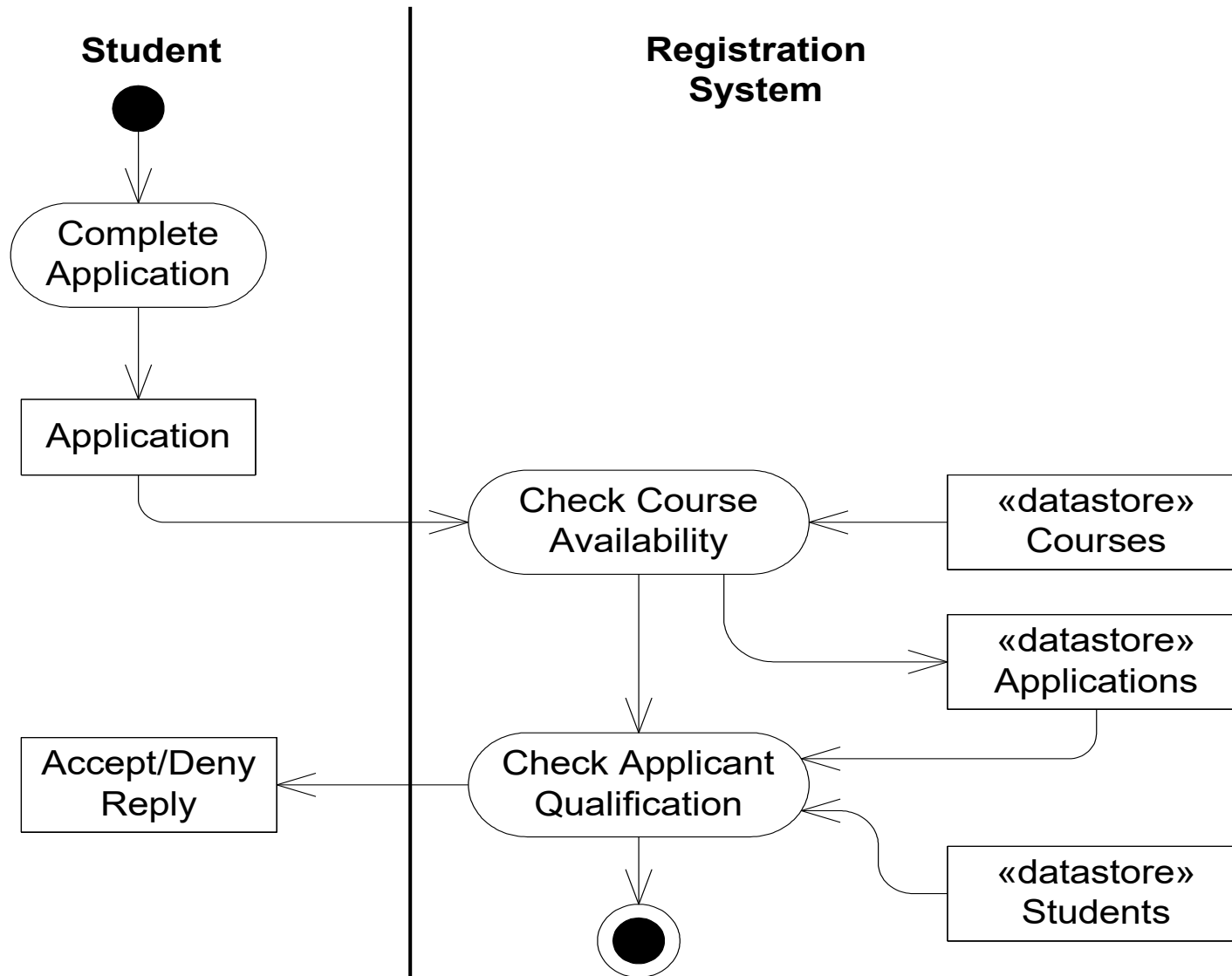
- Business Process Modeling
- Data Flow Modeling
- Concurrent programming and Parallel algorithm Modeling

Business process modeling

Leave Request Approval

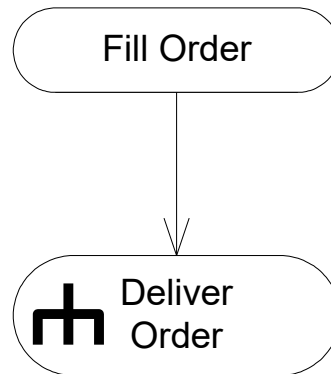


Data flow modeling



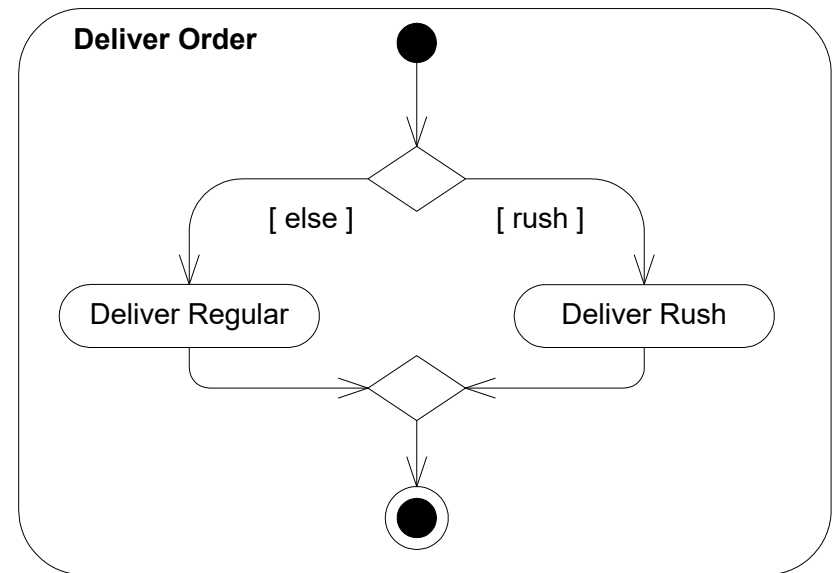
Complex activity diagrams

the “rake” symbol (which represents a hierarchy) indicates this activity is expanded in a sub-activity diagram

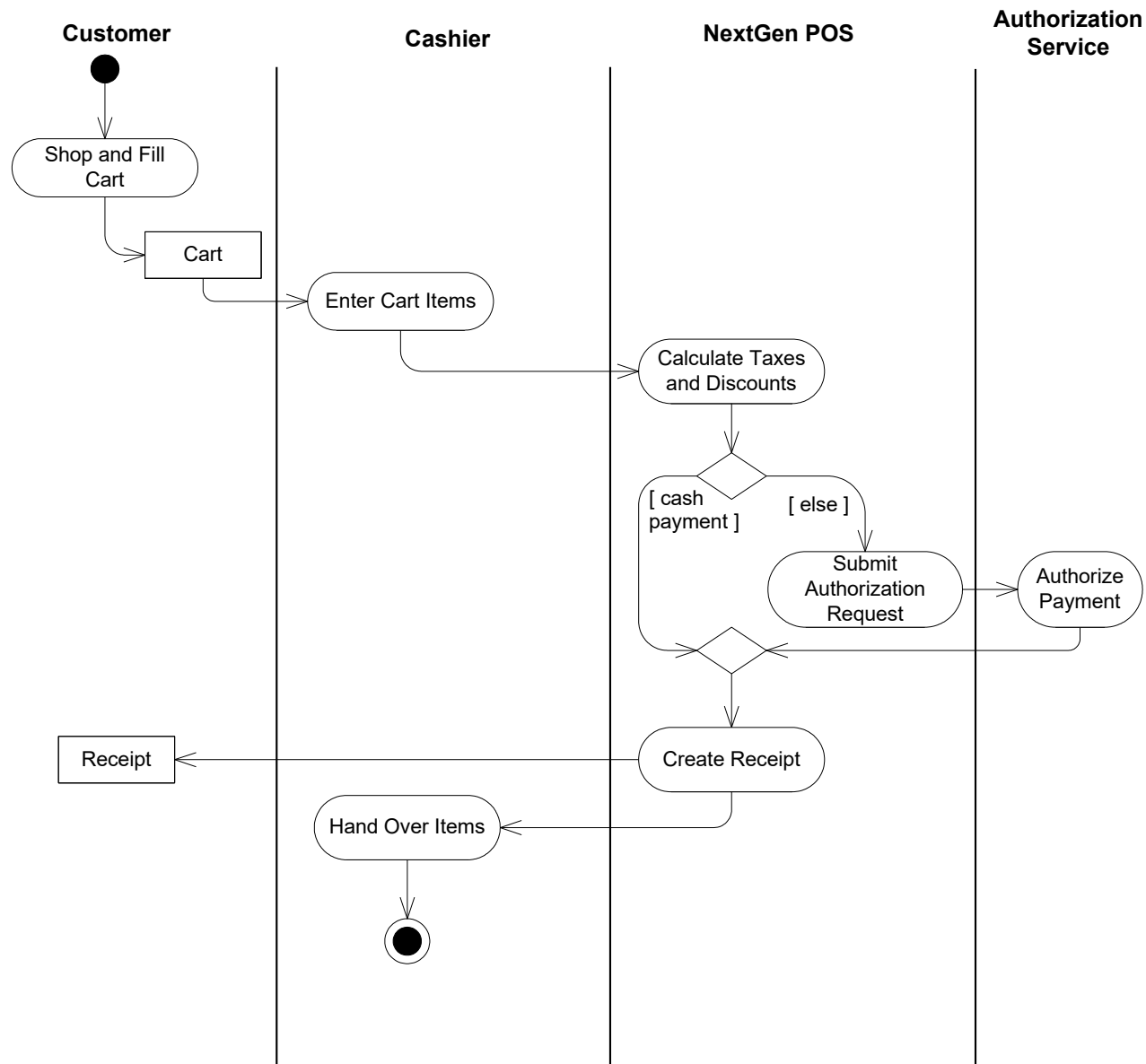


Decision: Any branch happens. Mutual exclusion

Merge: Any input leads to continuation. This is in contrast to a *join*, in which case *all* the inputs have to arrive before it continues.



Process sale activity diagram



From Workflow to Architecture

Activity Diagrams:

- Visualize complex processes clearly.
- Expose concurrency and dependencies.
- Reveal boundaries between responsibilities
→ candidate components.

Behavioral Element

Activity/Action

Swimlane

Control flow

Architectural Artifact

Component responsibility

Subsystem or component
boundary

Connector between
components

Thank you !